



USAL
UNIVERSIDAD
DEL SALVADOR

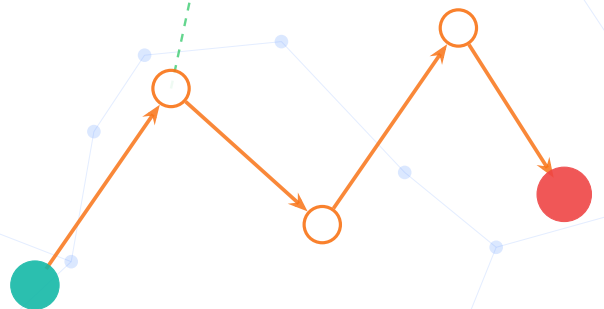
FACULTAD DE INGENIERÍA
PROYECTO FINAL DE INGENIERÍA INFORMÁTICA

GraphSAST: analizador estático de código basado en grafos de flujo de datos para detectar vulnerabilidades en aplicaciones JavaScript

Maximo Zuidwijk
m.zuidwijk@usal.edu.ar

PROFESOR
Esteban Tissera
Profesor, Facultad de Ingeniería

CICLO LECTIVO 2026



Este trabajo es original y fue elaborado exclusivamente con este fin. Todos los autores cuyos estudios contribuyeron a su elaboración se encuentran debidamente citados. Se permite su reproducción parcial con indicación del autor, del grado, del año académico, de la institución (Universidad del Salvador) y de la fecha de defensa pública.



Agradecimientos

[Texto de los agradecimientos.]

Resumen

GraphSAST es un sistema de análisis estático de código fuente (SAST) orientado a la detección temprana de vulnerabilidades de arquitectura de software. El sistema compila la lógica de aplicaciones escritas en TypeScript/JavaScript en una representación formal mediante un grafo de flujo de datos. A partir de técnicas de análisis de contaminación (taint analysis), la herramienta rastrea el recorrido de datos no confiables (sources) hacia operaciones sensibles (sinks) y verifica la presencia o ausencia de mecanismos de sanitización en el trayecto. El proyecto abarca el diseño e implementación propios del analizador, la construcción del grafo, el motor de consultas y una interfaz web interactiva para la visualización de los caminos de riesgo, validando su precisión y su capacidad de reducción de falsos positivos frente a repositorios con vulnerabilidades documentadas.

Palabras clave: análisis estático, análisis de contaminación, grafo de flujo de datos, seguridad de aplicaciones

Abstract

GraphSAST is a static application security testing (SAST) system aimed at the early detection of software architecture vulnerabilities. The system compiles the logic of applications written in TypeScript/JavaScript into a formal representation based on a data flow graph. Using taint analysis techniques, the tool traces the path of untrusted data (sources) towards sensitive operations (sinks) and checks whether sanitization mechanisms are present along the way. The project covers the design and implementation of the analyzer, the construction of the graph, the query engine and an interactive web interface for visualizing risk paths, and validates its precision and its ability to reduce false positives against repositories with documented vulnerabilities.

Keywords: static analysis, taint analysis, data flow graph, application security



Declaración sobre el uso de inteligencia artificial

Se reconoce el uso de Claude (Anthropic), a través de la herramienta Claude Code, como asistente durante el desarrollo del proyecto. Las tareas asistidas fueron la programación y las pruebas del prototipo, la revisión de la consistencia entre el documento y el código, la redacción de borradores de los capítulos, que luego fueron revisados y reescritos por el autor, y la conversión del documento al formato de la cátedra. Los cambios de código asistidos quedan identificados en el historial del repositorio mediante la línea Co-Authored-By. [Otras herramientas utilizadas y tareas asistidas.]

Todas las salidas fueron revisadas y verificadas por el autor, que asume la responsabilidad total sobre el contenido de este documento. Las cifras y referencias se contrastaron con la ejecución del sistema y con las fuentes originales. Los prompts relevantes se registran en [carpeta compartida con la cátedra].

Índice general

Agradecimientos	III
Resumen	IV
Abstract	V
Declaración sobre el uso de inteligencia artificial	VI
Índice de figuras	XII
Índice de tablas	XIII
1 Introducción	1
1.1 Introducción y contexto	1
1.2 Definición del problema	1
1.3 Objetivo	2
1.4 Barreras	2
1.5 Relevancia del proyecto	2
1.6 Hipótesis de investigación	3
1.7 Enfoque	3
1.8 Recursos	3
2 Metodología y procedimientos	4
2.1 Metodología de la investigación	4
2.1.1 Área temática	4
2.1.2 Planteamiento del problema	4
2.1.3 Delimitación de la investigación	4
2.2 Marco teórico	4
2.3 Diseño concreto de investigación	5
2.3.1 Indicadores	5
2.3.2 Técnicas de recolección de datos	5
2.3.3 Instrumentos de recolección de datos	5
2.3.4 Datos	5
2.3.5 Procesamiento de los datos	5
2.3.6 Análisis de los datos	5
2.3.7 Síntesis y conclusiones	6
2.4 Detalle del proyecto	6
2.4.1 Alcance	6
2.4.2 Tiempos	6
2.4.3 Costos	6
2.4.4 Riesgos	7
2.4.5 Calidad	7
2.4.6 Recursos	7
2.4.7 Satisfacción	7

3	Síntesis de la literatura consultada	8
3.1	Estado del Arte	8
3.1.1	Investigación Nacional	8
3.1.2	Brechas Identificadas en la Investigación Nacional	9
3.1.3	Investigaciones Internacionales	9
3.1.4	Productos Similares	10
3.2	Marco Teórico	11
3.2.1	Análisis Estático de Código Fuente (SAST) y «Building Security In»	12
3.2.2	Compiladores, Análisis Léxico, Parsing y Árboles de Sintaxis Abstracta (AST)	12
3.2.3	Representación del Código mediante Grafos: Call Graph y Data Flow Graph (DFG)	12
3.2.4	Análisis de Contaminación (Taint Analysis)	13
3.2.5	Bases de Datos Orientadas a Grafos y Motor de Consultas (Neo4j y Cypher)	14
3.2.6	Catálogo de Vulnerabilidades de Arquitectura (CWE)	14
3.2.7	Visualización de Software e Interfaz Interactiva de Grafos	15
3.3	Marco Conceptual	15
3.3.1	Vulnerabilidad de Arquitectura	15
3.3.2	Deuda Técnica de Seguridad	15
3.3.3	Indicadores de Calidad del Analizador: Precisión, Exhaustividad, Falsos Positivos y Negativos	15
3.3.4	Accesibilidad (Reachability) en Grafos Dirigidos	16
3.3.5	Sensibilidad en el Análisis de Programas	16
4	Marco tecnológico	17
4.1	Arquitectura del sistema	17
4.1.1	Organización en paquetes	17
4.1.2	Flujo de procesamiento	17
4.2	Prototipo y alcance de lo construido	18
4.2.1	Estado del desarrollo	18
4.2.2	Funcionalidades implementadas	18
4.2.3	Catálogo de detección	19
4.2.4	Verificación y resultados	19
4.2.5	Limitaciones del alcance actual	24
4.2.6	Próximos pasos	24
4.3	Stack de software	24
4.3.1	Estrategia de análisis sintáctico	25
4.3.2	Lenguaje del motor	25
4.3.3	Visualización	26
4.3.4	Herramientas de construcción y pruebas	26
4.4	Infraestructura y modelo de ejecución	26
4.4.1	Modelo de ejecución	26
4.4.2	Requerimientos	26
4.4.3	Disponibilidad y escalabilidad	27
4.5	Modelo de datos: el grafo del programa	27
4.5.1	Nodos	27
4.5.2	Aristas	27
4.5.3	Resolución de la detección sobre el modelo	28
4.6	Diseño e interacción	28

4.6.1	Usuario destinatario	28
4.6.2	Organización de la interfaz	28
4.6.3	Decisiones de diseño	29
4.6.4	Vinculación con el código fuente	30
4.6.5	Accesibilidad y validación	30
4.7	Calidad, seguridad y despliegue	30
4.7.1	Estrategia de pruebas	30
4.7.2	Control de versiones	30
4.7.3	Consideraciones de seguridad	30
4.7.4	Despliegue	31
5	Justificación económica	32
5.1	Escenario de comercialización	32
5.1.1	Suscripción frente a licencia perpetua	32
5.1.2	Público objetivo	32
5.1.3	Relación entre público y precio	33
5.1.4	Dimensionamiento del mercado	33
5.2	Inversión inicial	34
5.2.1	Supuestos de la estimación de horas	34
5.2.2	Determinación del valor hora	34
5.2.3	Composición de la inversión inicial	34
5.3	Costos post-proyecto	35
5.4	Flujo de fondos proyectado	36
5.4.1	Supuestos del flujo	36
5.5	Tasa de descuento	37
5.6	Indicadores financieros	38
5.6.1	Lectura de los indicadores	38
5.7	Análisis de sensibilidad y escenarios	38
5.7.1	Sensibilidad del Valor Actual Neto a la tasa de descuento	38
5.7.2	Escenarios de adopción	39
5.8	Conclusión	40
5.8.1	Condición crítica	40
5.8.2	Magnitud de la Tasa Interna de Retorno	40
5.8.3	Limitaciones del modelo	40
5.8.4	Conclusión general	40
6	Análisis de riesgos	41
6.1	Enfoque y escalas de evaluación	41
6.2	Supuestos y restricciones	42
6.3	Matriz de riesgos	43
6.4	Tratamiento de los riesgos principales	47
6.4.1	R-01: alcance del motor frente a aplicaciones reales	47
6.4.2	R-02: corpus de vulnerabilidades reales	47
6.4.3	R-03: alcance que no cierra	47
6.4.4	R-04: falsos positivos por coincidencia de nombres	48
6.4.5	R-05: dependencias con vulnerabilidades conocidas	48
6.4.6	R-06: participantes para la validación de H3	48
6.4.7	R-07: competencia de herramientas gratuitas	48
6.4.8	R-08: mantenimiento del catálogo	49

6.5	Riesgos críticos de un proyecto final	49
6.6	Continuidad del sistema después de la entrega	50
6.7	Seguimiento y actualización	50
7	Presentación de resultados	52
7.1	Requerimientos del sistema	52
7.2	Proceso de desarrollo	54
7.3	Evidencia de implementación	55
7.4	Pruebas técnicas	56
7.4.1	Pruebas automatizadas	56
7.4.2	Banco de pruebas sintético	56
7.4.3	Comparación con una línea de base	57
7.4.4	Vulnerabilidades documentadas en paquetes de npm	58
7.4.5	Aplicación deliberadamente vulnerable: OWASP NodeGoat	58
7.4.6	Rendimiento	58
7.5	Validación con usuarios y con un experto	58
7.6	Resultados desfavorables y su análisis	59
7.7	Síntesis de la evidencia	60
8	Implicancias, conclusiones y recomendaciones	61
8.1	Respuesta a las hipótesis	61
8.2	Cumplimiento de los objetivos	62
8.3	Implicancias	62
8.4	Limitaciones	62
8.5	Recomendaciones	63
8.6	Trabajo futuro	63
8.7	Contribución al conocimiento	63
	Bibliografía	64
	Apéndices	67
	Apéndice A Encuesta de validación	68
A.1	Introducción	68
A.2	Metodología	68
A.3	Presentación de la encuesta	69
A.4	Cuestionario	69
A.5	Casos utilizados	69
	Apéndice B Entrevista a [nombre y título de la persona entrevistada]	71
B.1	Contexto de la entrevista	71
B.2	Datos de la persona entrevistada	71
B.3	Transcripción de la entrevista	71
B.3.1	[Tema 1]	71
B.3.2	[Tema 2]	71
	Apéndice C Detalles técnicos e implementación	72
C.1	Versiones	72
C.2	Parámetros del análisis	72
C.3	Catálogo de detección	72

C.4 Reproducción de los resultados	73
C.5 Variables de entorno	73
Anexos	74
Anexo I Repositorios de código fuente	75
I.1 GraphSAST	75
I.2 OWASP NodeGoat	75
Glosario	76
Siglas	78
Símbolos	79

Índice de figuras

3.1	Flujo general de procesamiento de GraphSAST: código fuente, parser, grafo de flujo de datos, consultas y visualización	13
3.2	Recorrido source-sink utilizado en el análisis de taint	14
4.1	Arquitectura de GraphSAST: pipeline de análisis por etapas, desde los archivos del proyecto hasta la visualización y los reportes, con enlace entre archivos	18
4.2	Detección de vulnerabilidad (CWE-89): el dato fluye desde el parámetro de entrada hasta la operación de consulta sin saneamiento intermedio. El camino de riesgo se resalta en la visualización	20
4.3	Caso saneado: el mismo patrón de flujo con una operación de saneamiento intermedia. El sistema identifica el origen y el destino sensible, pero descarta el hallazgo al constatar el saneamiento en el recorrido	21
4.4	Panel de métricas y líneas del camino de riesgo correspondientes al análisis ejecutado	21
4.5	Versión web: análisis de un proyecto subido como carpeta. El hallazgo cruza tres archivos y se muestra el archivo del sink con las líneas del camino resaltadas	29
5.1	Flujo de fondos neto y acumulado, en dólares (Años 0 a 5). El acumulado se vuelve positivo a los 1,94 años	37
5.2	Valor Actual Neto según la tasa de descuento anual	39

Índice de tablas

2.1	Indicadores de evaluación del prototipo	5
2.2	Cronograma de desarrollo e implementación	6
2.3	Costos estimados del proyecto	6
2.4	Riesgos principales y mitigación	7
3.1	Comparación de herramientas SAST similares	11
4.1	Debilidades cubiertas por el catálogo de detección implementado	19
4.2	Resultados del banco de pruebas sintético (43 casos; tiempos: mediana de cinco ejecuciones)	22
4.3	Validación sobre vulnerabilidades documentadas del ecosistema npm (6 casos)	23
4.4	Tecnologías empleadas en el desarrollo del prototipo	25
4.5	Categorías de nodo del modelo de grafo	27
4.6	Categorías de relación del modelo de grafo	28
5.1	Supuestos de estimación de horas de desarrollo	34
5.2	Presupuesto de inversión inicial (Año 0)	35
5.3	Costos operativos anuales post-proyecto, en dólares (Años 1 a 5)	36
5.4	Flujo de fondos proyectado, en dólares (Años 0 a 5)	36
5.5	Resultados de los indicadores financieros	38
5.6	Sensibilidad del Valor Actual Neto frente a variaciones en la tasa de descuento	39
5.7	Evaluación de escenarios sobre la base de usuarios	39
6.1	Matriz de exposición: probabilidad por impacto	42
6.2	Supuestos (S) y restricciones (C) del proyecto	43
6.3	Matriz de riesgos ordenada por exposición. P: probabilidad; I: impacto; E: exposición	44
6.4	Relación entre los riesgos críticos de un proyecto final y los riesgos identificados	50
7.1	Requerimientos funcionales	53
7.2	Requerimientos no funcionales	54
7.3	Etapas de desarrollo completadas	55
7.4	Pruebas automatizadas por etapa	56
7.5	Composición del banco sintético por familia	57
7.6	Matriz de confusión sobre el banco sintético (43 casos)	57
7.7	Resultados de la línea de base y de GraphSAST sobre los mismos casos	57
7.8	Resultados por condición en la validación con usuarios	59
7.9	Traducción de la retroalimentación en decisiones de diseño	59
7.10	Evidencia obtenida por hipótesis	60
8.1	Grado de cumplimiento de los objetivos específicos	62
A.1	Órdenes de presentación de las condiciones y los casos	68

C.1 Versiones de las dependencias principales 72

C.2 Parámetros del motor con sus valores por defecto 72

C.3 Variables de entorno 73

1.1. Introducción y contexto

El desarrollo de software moderno exige la incorporación de controles de seguridad en las etapas iniciales del ciclo de vida (enfoque *shift-left*). Tradicionalmente, la auditoría de seguridad se realiza mediante análisis dinámico en fases avanzadas de prueba o a través de analizadores estáticos basados en coincidencia de patrones de texto y firmas de vulnerabilidades conocidas.

La limitación de las herramientas basadas en patrones es que no interpretan la semántica del programa. Tienden a marcar variables como inseguras simplemente por su nombre, o bien pierden el rastro de los datos cuando estos atraviesan varias funciones, desestructuraciones o módulos. Las fallas de seguridad más difíciles de detectar no suelen ser un comando peligroso aislado en una línea, sino un problema de diseño en el modo en que la información viaja entre los componentes del sistema.

Para resolver esto, GraphSAST convierte el código fuente en un grafo dirigido que modela las llamadas, dependencias y mutaciones de las variables. De esta manera, auditar la seguridad se convierte en un problema computable de accesibilidad sobre el grafo.

1.2. Definición del problema

Las herramientas SAST convencionales presentan dificultades concretas en proyectos reales:

Falsos positivos por falta de contexto. Al buscar firmas o patrones de texto, alertan sobre variables que en realidad ya fueron validadas o que no reciben datos externos, y saturan al equipo con reportes que requieren revisión manual.

Pérdida de trazabilidad en flujos complejos. Cuando una entrada de usuario pasa por middlewares, se asigna a propiedades de objetos o se transfiere entre módulos, los analizadores por expresiones regulares pierden el recorrido del dato.

Dificultad para interpretar reportes planos. Las listas tradicionales de advertencias muestran la línea del fallo, pero obligan al programador a reconstruir mentalmente todo el camino que siguió la variable desde su origen.

Estas limitaciones definen la pregunta de investigación: ¿puede un analizador basado en grafos de flujo de datos detectar vulnerabilidades de arquitectura con mayor precisión y menor cantidad de falsos positivos que las herramientas basadas en coincidencia de patrones de texto?

1.3. Objetivo

Objetivo general: Desarrollar un sistema de análisis estático de código fuente (SAST) que compile la lógica de una aplicación en un grafo de flujo de datos para auditar, detectar y visualizar de forma interactiva fallas de arquitectura de software y posibles fugas de información, evaluando su precisión sobre proyectos reales.

Objetivos específicos.

- OE-1. Construir el analizador sintáctico que procese código fuente TypeScript/JavaScript mediante la biblioteca ts-morph y extraiga los nodos relevantes del Árbol de Sintaxis Abstracta (AST).
- OE-2. Diseñar e implementar el algoritmo de transformación del AST en un grafo dirigido de flujo de datos e invocaciones, con aristas FLOWS_TO, CALLS, BINDS_TO, RETURNS y SANITIZED_BY.
- OE-3. Desarrollar el motor de análisis de taint que detecte caminos source-sink no sanitizados para los patrones CWE-89, CWE-79 y CWE-78, parametrizado mediante un catálogo externo en formato JSON.
- OE-4. Persistir el grafo compilado en la base de datos Neo4j y exponer las consultas de accesibilidad mediante el lenguaje Cypher.
- OE-5. Construir la interfaz web interactiva (TypeScript con Cytoscape.js) que visualice la topología del programa y resalte los recorridos de riesgo detectados.
- OE-6. Validar el prototipo contra un banco de pruebas de casos sintéticos y proyectos reales, midiendo precisión, exhaustividad, falsos positivos y falsos negativos frente a una herramienta de referencia.

1.4. Barreras

- Complejidad del análisis de lenguajes reales: la construcción de un analizador robusto para un lenguaje moderno implica el tratamiento de numerosas construcciones sintácticas.
- Escalabilidad del grafo: el tamaño del grafo crece con el código, por lo que el mantenimiento de tiempos de cómputo razonables constituye un desafío.
- Precisión del análisis de taint: la reducción de falsos positivos sin perder vulnerabilidades reales requiere un modelado cuidadoso de fuentes, destinos y sanitizadores.
- Alcance del lenguaje: la cobertura de todas las construcciones de un lenguaje resulta inviable en el tiempo del proyecto, por lo que debe acotarse a un subconjunto documentado.

1.5. Relevancia del proyecto

Las vulnerabilidades de seguridad conllevan un alto costo. Una herramienta capaz de detectar fallas de arquitectura antes de la ejecución, con menor ruido que los analizadores

tradicionales y con una visualización comprensible, aporta valor directo a los equipos de desarrollo. En el plano académico, el proyecto integra teoría de grafos, análisis de lenguajes de programación y ciberseguridad aplicada, y produce un artefacto verificable sobre código real.

El trabajo se vincula asimismo con las disciplinas de aseguramiento de la calidad del software (Quality Assurance). Mientras las pruebas funcionales validan el comportamiento dinámico del sistema en ejecución, el análisis estático mediante grafos ofrece un control preventivo sobre el diseño y la arquitectura en una etapa previa a la compilación y el despliegue, lo que sitúa a GraphSAST como complemento del testing tradicional dentro del ciclo de aseguramiento de la calidad.

1.6. Hipótesis de investigación

H1. La representación del código fuente como un grafo de flujo de datos permite identificar vulnerabilidades de arquitectura y recorridos de riesgo con mayor precisión que los analizadores basados en coincidencias aisladas de patrones de texto.

H2. El análisis de taint estructurado sobre el grafo reduce la tasa de falsos positivos respecto de la detección basada en expresiones regulares.

H3. La visualización topológica del recorrido del dato facilita la interpretación del riesgo por parte del equipo de desarrollo frente a los listados de errores en texto plano.

1.7. Enfoque

El proyecto se enmarca en una investigación aplicada con validación empírica. El prototipo se construye de manera incremental (analizador → grafo → consultas → visualización) y se evalúa cuantitativamente contra un banco de pruebas compuesto por casos sintéticos con fallas conocidas y proyectos reales de código abierto con vulnerabilidades documentadas, midiendo precisión, exhaustividad, falsos positivos, falsos negativos y tiempo de análisis.

1.8. Recursos

- **Software:** biblioteca de parsing ts-morph, base de datos de grafos Neo4j, biblioteca de visualización Cytoscape.js, entorno de testing Vitest y control de versiones Git.
- **Datos:** repositorios públicos con vulnerabilidades documentadas y casos sintéticos contruidos para la validación.
- **Hardware:** equipo de desarrollo personal.
- **Humanos:** el alumno como responsable del desarrollo y la validación.

2.1. Metodología de la investigación

2.1.1. Área temática

Ciberseguridad aplicada y análisis de programas: análisis estático de código (SAST), representación del programa mediante grafos y análisis de flujo de datos (taint analysis).

2.1.2. Planteamiento del problema

¿Puede un analizador basado en grafos de flujo de datos detectar vulnerabilidades de arquitectura con mayor precisión y menos falsos positivos que las herramientas basadas en coincidencia de patrones de texto?

2.1.3. Delimitación de la investigación

El motor se enfoca en el lenguaje base TypeScript/JavaScript y en tres patrones de riesgo definidos según el catálogo CWE (CWE-89 Inyección SQL, CWE-79 Cross-Site Scripting y CWE-78 Inyección de Comandos). No se incluyen la corrección automática de código ni la integración como complemento (plugin) de entornos de desarrollo comerciales.

2.2. Marco teórico

El proyecto se apoya en la teoría de grafos, en los métodos de compilación (análisis sintáctico y árboles de sintaxis abstracta) y en el análisis de flujo de control y de datos. El concepto central es el análisis de taint: el seguimiento de los datos no confiables desde su punto de entrada (source) hasta los puntos críticos donde se utilizan (sink), verificando si fueron sanitizados en el trayecto. Bajo este planteo, la seguridad se formaliza como un problema de accesibilidad sobre un grafo dirigido: existe una vulnerabilidad cuando hay un camino desde un source hasta un sink que no atraviesa un sanitizer.

2.3. Diseño concreto de investigación

2.3.1. Indicadores

Tabla 2.1: Indicadores de evaluación del prototipo

Indicador	Definición operativa	Medición prevista
Precisión (Precision)	Proporción de vulnerabilidades reales sobre el total de alertas emitidas.	$TP / (TP + FP)$
Exhaustividad (Recall)	Proporción de vulnerabilidades reales detectadas sobre el total existente.	$TP / (TP + FN)$
Falsos positivos (FP)	Alertas generadas sobre código seguro o sanitizado.	Conteo de instancias marcadas erróneamente.
Falsos negativos (FN)	Vulnerabilidades reales presentes que el sistema no detecta.	Conteo de omisiones.
Tiempo de análisis	Tiempo de cómputo para procesar el AST, construir el grafo y ejecutar las consultas por cada 1.000 líneas de código (KLOC).	Medición en segundos mediante scripts de benchmarking.

2.3.2. Técnicas de recolección de datos

Selección de repositorios públicos con vulnerabilidades documentadas y construcción de casos sintéticos con fallas controladas.

2.3.3. Instrumentos de recolección de datos

Banco de pruebas versionado, scripts de evaluación automatizada y planilla de registro de resultados por caso.

2.3.4. Datos

Código fuente de los proyectos de prueba, etiquetas de las vulnerabilidades esperadas y mediciones de desempeño del analizador.

2.3.5. Procesamiento de los datos

Ejecución del analizador sobre cada caso, generación del grafo, ejecución de las consultas de detección y registro de aciertos, falsos positivos y falsos negativos.

2.3.6. Análisis de los datos

Cálculo de las métricas de desempeño, comparación contra la herramienta de referencia y análisis de los tiempos de cómputo en función del tamaño del código.

2.3.7. Síntesis y conclusiones

Se evaluarán las hipótesis H1-H3 a partir de los resultados, determinando si el enfoque por grafos mejora la detección y reduce el ruido respecto del análisis por patrones.

2.4. Detalle del proyecto

2.4.1. Alcance

Incluye: motor para el lenguaje base TypeScript/JavaScript, detección de tres patrones de riesgo arquitectónico (CWE-89, CWE-79 y CWE-78), persistencia del grafo, visualización interactiva y banco de pruebas con evaluación reproducible.

No incluye: corrección o refactorización automática del código, ni integración nativa como complemento de entornos de desarrollo comerciales.

2.4.2. Tiempos

El cronograma se organiza en etapas de desarrollo, validación y redacción, ajustables al calendario académico.

Tabla 2.2: Cronograma de desarrollo e implementación

Etapa / Hito	Entregable	Período estimado
Hito 1	Analizador y extracción del Árbol de Sintaxis Abstracta (AST).	Mes 1-2
Hito 2	Algoritmo de construcción del grafo de flujo de datos.	Mes 3-4
Hito 3	Motor de consultas (taint analysis) e integración con la base de datos de grafos.	Mes 5-6
Hito 4	Interfaz web interactiva de visualización.	Mes 7-8
Hito 5	Validación con proyectos reales, benchmarking y redacción de la memoria final.	Mes 9-10

2.4.3. Costos

El proyecto se plantea como un prototipo académico de bajo costo, priorizando herramientas libres, documentación pública y recursos propios.

Tabla 2.3: Costos estimados del proyecto

Recurso	Tipo	Costo estimado
Hardware	Equipo de cómputo personal de desarrollo.	Sin costo adicional
Software y base de datos	ts-morph, Neo4j Community, Cytoscape.js, Vite, Vitest.	Licencias open source / gratuitas
Datos de prueba	Repositorios públicos (OWASP, NIST SARD, GitHub).	Acceso público sin costo
Total estimado	—	USD 0,00

2.4.4. Riesgos

Tabla 2.4: Riesgos principales y mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Complejidad excesiva en el análisis del lenguaje	Media	Alto	Emplear bibliotecas de parsing maduras y acotar las construcciones sintácticas soportadas a un subconjunto documentado.
Degradación del rendimiento al escalar el grafo	Media	Medio	Optimizar los índices en la base de datos, acotar la profundidad de búsqueda en los recorridos y limitar el análisis de dependencias externas.
Generación elevada de falsos positivos	Media	Alto	Refinar la definición de sources, sinks y sanitizadores en el catálogo, e incorporar verificaciones de contexto en el motor de consultas.

2.4.5. Calidad

La calidad se asegura mediante un banco de pruebas versionado, una evaluación automatizada y reproducible, y pruebas unitarias del software (Vitest) por módulo.

2.4.6. Recursos

Humanos: el alumno. Tecnológicos: biblioteca de parsing (ts-morph), base de datos de grafos (Neo4j), biblioteca de visualización (Cytoscape.js) y entorno de testing (Vitest). Materiales: equipo de desarrollo personal.

2.4.7. Satisfacción

El grado de satisfacción se mide por la precisión alcanzada y la reducción de falsos positivos respecto de la herramienta de referencia, y por la claridad de la visualización para identificar el recorrido del riesgo.

3.1. Estado del Arte

3.1.1. Investigación Nacional

En el ámbito académico nacional existen antecedentes directos en análisis estático de programas y, en particular, en análisis de contaminación (taint analysis) aplicado a JavaScript. La producción más relevante proviene del Departamento de Computación de la Facultad de Ciencias Exactas y Naturales de la Universidad de Buenos Aires (FCEyN-UBA) y del Instituto de Investigación en Ciencias de la Computación (ICC, UBA-CONICET), cuyas líneas de investigación se orientan al análisis automático de programas para su comprensión, verificación y validación.

Dentro de esa línea de trabajo, Balbi (2023) desarrolla una investigación centrada en la inferencia automática de especificaciones de taint analysis para el lenguaje JavaScript. El trabajo parte de la premisa de que la efectividad del análisis de contaminación no depende únicamente del motor de rastreo, sino de la calidad de las especificaciones que definen qué elementos del programa constituyen puntos de entrada (sources) o de riesgo (sinks). La solución propuesta combina un método basado en aprendizaje automático con el motor de análisis estático CodeQL, empleado como infraestructura de consulta subyacente.

Esta investigación se articula con producción científica de proyección internacional. Dutta et al. (2022) presentan InspectJS en el 44th International Conference on Software Engineering (ICSE-SEIP), trabajo que aborda la inferencia de especificaciones de taint para JavaScript mediante similitud de código y retroalimentación del usuario. Los autores sostienen que el mecanismo de rastreo de contaminación se implementa una única vez en el motor y luego se parametriza con distintos conjuntos de sources, sinks y sanitizers, lo que permite detectar categorías diversas de vulnerabilidades sin modificar la lógica interna del analizador. Este principio de desacoplamiento entre el motor de análisis y el catálogo de reglas constituye un fundamento directo del diseño de catálogo externo en formato JSON adoptado en el presente proyecto.

En el plano de la investigación aplicada, se registran trabajos nacionales que emplean el análisis estático como instrumento de auditoría sobre productos concretos. La Universidad Argentina de la Empresa (UADE) desarrolló una investigación orientada a evaluar las características de seguridad de videojuegos basados en NFT Gaming (2021-2022), combinando el análisis estático del código fuente en busca de vulnerabilidades con el análisis dinámico de los datos recopilados por las aplicaciones (Villan & Melani, 2022).

Como ámbito de difusión científica, el Simposio Argentino de Ingeniería de Software (ASSE), integrado a las Jornadas Argentinas de Informática (JAIIO) organizadas por la Sociedad Argentina de Informática e Investigación Operativa (SADIO) desde 1961, constituye el foro nacional de referencia para la producción en ingeniería de software y ha sido consultado como fuente de antecedentes locales.

3.1.2. Brechas Identificadas en la Investigación Nacional

El relevamiento de la literatura nacional permite delimitar con precisión el aporte del presente proyecto:

Dependencia de motores industriales preexistentes: los antecedentes nacionales de investigación fundamental operan sobre infraestructuras de análisis ya construidas (particularmente CodeQL) y concentran su innovación en la capa de inferencia de especificaciones. La construcción propia del recorrido posterior al análisis sintáctico —la transformación del árbol en grafo de flujo de datos, su persistencia y el motor de consultas autónomo— permanece fuera del alcance de estos trabajos, que la asumen como un componente disponible.

Ausencia de implementaciones académicas reproducibles de extremo a extremo: no se identifican en el ámbito nacional desarrollos académicos de código abierto que documenten e implementen de manera integral la compilación directa del código fuente a un grafo de propiedades persistido, junto con el motor de consultas de accesibilidad y un banco de pruebas reproducible por terceros.

Orientación aplicada desacoplada del diseño del analizador: los trabajos de investigación aplicada locales emplean herramientas comerciales o de código abierto existentes como instrumentos de auditoría sobre software final, sin abordar el diseño algorítmico del motor de detección ni la evaluación cuantitativa de su precisión frente a enfoques alternativos.

Escasa exploración de la visualización topológica del riesgo: la representación interactiva del recorrido del dato contaminado sobre la topología del programa, orientada a reducir el esfuerzo de interpretación del hallazgo por parte del equipo de desarrollo, no aparece tratada como objeto de estudio en los antecedentes nacionales relevados.

3.1.3. Investigaciones Internacionales

A nivel internacional, la representación del código mediante grafos y el análisis de flujo de datos cuentan con antecedentes teóricos y empíricos fundamentales:

Code Property Graphs (CPG): Yamaguchi et al. (2014) formalizaron la representación unificada del código fuente combinando en una misma estructura de grafo el Árbol de Sintaxis Abstracta (AST), el Grafo de Flujo de Control (CFG) y el Grafo de Dependencia de Datos (DDG). Este trabajo mostró que la detección de vulnerabilidades puede expresarse como un problema de consulta de accesibilidad sobre grafos.

Object Dependence Graphs (ODG): Li et al. (2022) desarrollaron un modelo basado en grafos de dependencia de objetos para detectar vulnerabilidades en el ecosistema Node.js / JavaScript, demostrando que el rastreo de objetos en lenguajes dinámicos permite abordar patrones de riesgo no detectados por el análisis estático convencional.

Estudios comparativos de herramientas SAST en Node.js: Brito et al. (2023) evaluaron empíricamente la efectividad de herramientas de análisis estático sobre paquetes reales del ecosistema Node.js, caracterizando sus limitaciones de cobertura y precisión.

3.1.4. Productos Similares

En la industria del software existen herramientas consolidadas que aplican principios de análisis de flujo de datos e inspección de código:

Tabla 3.1: Comparación de herramientas SAST similares

Herramienta	Motor de Análisis	Alcance y Enfoque	Diferenciación con GraphSAST
CodeQL (GitHub / Semmle)	Motor relacional basado en grafos interpretados con lenguaje declarativo QL.	Plataforma profesional multilenguaje de alcance industrial.	Requiere infraestructura y compilación pesada; GraphSAST implementa un motor liviano enfocado en TypeScript/JavaScript con persistencia nativa en base de datos de grafos y visualización web interactiva.
Semgrep	Motor de coincidencia de patrones sintácticos abstractos con soporte de taint mode.	Análisis rápido en pipelines de CI/CD basado en reglas declarativas en YAML.	Su enfoque principal es la ejecución de reglas en línea de comandos; no persiste la topología completa del programa en una base de datos de grafos para consultas de arquitectura.
Joern	Plataforma de código abierto basada en Code Property Graphs (CPG).	Análisis estático avanzado enfocado en C/C++ y Java utilizando la base OverflowDB.	Orientado a análisis por consola mediante entorno REPL (Scala); GraphSAST simplifica la inspección mediante un panel web con renderizado dinámico de nodos.

3.2. Marco Teórico

En consonancia con la metodología académica establecida, se definen en primer lugar las variables de investigación y su relación teórica:

Variable independiente (V_1). representación del código fuente mediante grafos de flujo de datos (Data Flow Graphs).

Variable dependiente (V_2). detección estática de vulnerabilidades de arquitectura de software y precisión en el análisis de contaminación (Taint Analysis).

Relación entre las Variables: la hipótesis central de este trabajo sostiene que transformar la estructura sintáctica del código fuente en una representación matemática en forma de grafo dirigido permite formular la detección de vulnerabilidades como un problema formal de accesibilidad (reachability), mejorando la precisión y reduciendo la tasa de falsos positivos en comparación con los analizadores estáticos basados en coincidencia de patrones de texto.

3.2.1. Análisis Estático de Código Fuente (SAST) y «Building Security In»

El análisis estático de código fuente consiste en la evaluación del código de una aplicación sin requerir su ejecución dinámica (Nielson et al., 1999). En la ingeniería de software, McGraw (2006) formalizó el principio de incorporar prácticas de seguridad de manera intrínseca desde las fases iniciales del ciclo de vida del desarrollo de software (SDLC), enfoque que la industria denominaría posteriormente bajo el término Shift-Left Security. Detectar y corregir defectos de diseño y vulnerabilidades durante la fase de codificación evita que la deuda técnica de seguridad se propague hacia los entornos de prueba o producción, reduciendo los costos de remediación.

3.2.2. Compiladores, Análisis Léxico, Parsing y Árboles de Sintaxis Abstracta (AST)

La teoría clásica de compiladores establece que el procesamiento del código fuente comienza con la fase de análisis léxico (scanning), donde la secuencia lineal de caracteres se convierte en una serie de tokens, seguida por el análisis sintáctico (parsing), que construye una representación jerárquica denominada Árbol de Sintaxis Abstracta (Abstract Syntax Tree - AST) (Aho et al., 2006). El AST representa la estructura gramatical del programa en forma de árbol.

En el marco de este proyecto, el proceso de parsing se fundamenta en la API del compilador oficial de TypeScript a través de la biblioteca ts-morph. Esta herramienta permite consumir código fuente escrito en JavaScript/TypeScript y extraer una representación estructurada compuesta por nodos sintácticos (tales como declaraciones de funciones, asignaciones de variables, llamadas a métodos e identificadores).

3.2.3. Representación del Código mediante Grafos: Call Graph y Data Flow Graph (DFG)

Aunque el AST proporciona la estructura sintáctica del programa, no modela de forma explícita las relaciones de dependencia ni el recorrido de los datos a través de las diferentes funciones y módulos (Yamaguchi et al., 2014). Para superar esta limitación, la literatura propone construir dos representaciones sobre el AST:

Call Graph (Grafo de Invocaciones): grafo dirigido donde los nodos corresponden a funciones o métodos y las aristas dirigidas representan llamadas efectivas entre ellos, permitiendo realizar un análisis tanto intraprocedural (dentro de una misma función) como interprocedural (entre distintos archivos y módulos).

Data Flow Graph - DFG (Grafo de Flujo de Datos): grafo dirigido $G = (V, E)$ donde el conjunto de vértices V representa variables, operaciones e identificadores, y el conjunto de aristas dirigidas E modela la propagación y mutación de los valores.

En la fundamentación de este trabajo, el Grafo de Flujo de Datos formaliza las relaciones entre nodos mediante los siguientes tipos de aristas dirigidas:

- **FLows_TO:** indica la transferencia o asignación directa del valor de una variable a otra.
- **CALLS:** representa la invocación de una función o método con paso de parámetros.
- **BINDS_TO:** modela la vinculación semántica entre parámetros formales y argumentos reales.
- **RETURNS:** representa el retorno de un valor desde el ámbito de una función hacia el contexto invocador.
- **SANITIZED_BY:** modela la interceptación y neutralización del dato por parte de una función de validación o limpieza.



Figura 3.1: Flujo general de procesamiento de GraphSAST: código fuente, parser, grafo de flujo de datos, consultas y visualización

3.2.4. Análisis de Contaminación (Taint Analysis)

El análisis de contaminación (taint analysis) es un método formal fundamentado en la teoría del flujo de información (Denning, 1976; Nielson et al., 1999; Sabelfeld & Myers, 2003) para verificar si datos provenientes de fuentes no confiables pueden alcanzar operaciones sensibles sin una sanitización intermedia adecuada. El modelo teórico se define mediante la tupla (S, K, F) :

Source (S , origen). conjunto de nodos que representan puntos de entrada de información externa al programa no controlada por el sistema (por ejemplo, parámetros HTTP req.body, req.query, o entradas de usuario).

Sink (K , destino). conjunto de nodos que representan operaciones o funciones críticas donde la utilización de un dato no seguro genera una vulnerabilidad (por ejemplo, consultas SQL db.query(), ejecución de comandos child_process.exec(), o renderizado en interfaz).

Sanitizer (F , sanitizador). conjunto de nodos u operaciones que aplican filtros, codificaciones o validaciones que remueven la condición de contaminación del dato.

Desde la perspectiva de la teoría de grafos, una vulnerabilidad de seguridad se define formalmente como un problema de accesibilidad (reachability): existe una vulnerabilidad si y solo si en el grafo G existe un camino dirigido $p = (v_1, v_2, \dots, v_n)$ tal que $v_1 \in S$, $v_n \in K$ y $\forall v_i \in p$, $v_i \notin F$.

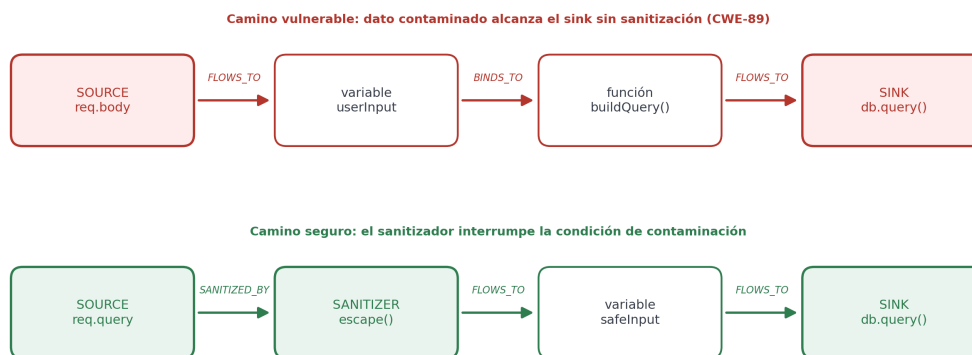


Figura 3.2: Recorrido source-sink utilizado en el análisis de taint

3.2.5. Bases de Datos Orientadas a Grafos y Motor de Consultas (Neo4j y Cypher)

Una base de datos orientada a grafos es un sistema de gestión de bases de datos diseñado para almacenar y consultar estructuras de red compuestas por nodos, aristas y propiedades (Robinson et al., 2013). Su ventaja técnica fundamental frente a las bases de datos relacionales en la ejecución de recorridos locales reside en la propiedad de adyacencia sin índices (index-free adjacency), donde cada nodo mantiene referencias directas hacia sus nodos vecinos, permitiendo explorar caminos sin realizar operaciones costosas de cruce (joins).

Para la aplicación en este proyecto, se adopta Neo4j como motor de persistencia del grafo compilado y el lenguaje Cypher como motor de consultas declarativo. Cypher permite expresar patrones de accesibilidad de longitud variable entre nodos de tipo Source y Sink, recorriendo de forma combinada las relaciones de flujo de datos, invocación, vinculación de parámetros y retorno, y descartando aquellos caminos en los que interviene una operación de sanitización.

3.2.6. Catálogo de Vulnerabilidades de Arquitectura (CWE)

La clasificación de vulnerabilidades se fundamenta en el estándar internacional Common Weakness Enumeration (CWE) de la organización MITRE (MITRE, s.f.). En este trabajo, el motor de detección se enfoca en tres patrones de riesgo arquitectónico:

CWE-89 (SQL Injection): ocurre cuando datos no sanitizados de una source fluyen hacia la construcción de consultas SQL dinámicas en un sink de base de datos.

CWE-79 (Cross-Site Scripting - XSS): ocurre cuando datos no confiables alcanzan salidas del navegador o respuestas HTTP sin la debida codificación.

CWE-78 (Command Injection): ocurre cuando entradas externas fluyen hacia funciones de ejecución de comandos del sistema operativo (exec, spawn).

3.2.7. Visualización de Software e Interfaz Interactiva de Grafos

La disciplina de visualización de software (Software Visualization) sostiene que la representación gráfica interactiva de las estructuras y flujos internos del programa reduce la carga cognitiva en la comprensión de la arquitectura y la identificación de fallas (Diehl, 2007). La integración de herramientas de renderizado de grafos como Cytoscape.js en entornos web modernos permite transformar los hallazgos de seguridad en diagramas interactivos donde los caminos críticos Source → Sink se resaltan sobre la topología del código, facilitando el diagnóstico por parte del desarrollador.

3.3. Marco Conceptual

3.3.1. Vulnerabilidad de Arquitectura

Defecto de seguridad cuya existencia no se manifiesta en una sentencia aislada del código, sino en la forma en que los componentes del sistema se vinculan entre sí y en el recorrido que la información describe a través de ellos. A diferencia de las vulnerabilidades localizadas —identificables mediante la inspección de una única línea o expresión—, una vulnerabilidad de arquitectura requiere reconstruir la propagación del dato a través de múltiples funciones, módulos o capas de abstracción para poder ser detectada. Su identificación exige, en consecuencia, una representación del programa que exceda el ámbito de la sentencia individual.

3.3.2. Deuda Técnica de Seguridad

Especialización del concepto de deuda técnica que cuantifica el costo diferido y el riesgo acumulado en un sistema de software debido a decisiones de diseño o deficiencias de seguridad no resueltas en el código fuente. Una alta deuda técnica de seguridad incrementa la superficie de ataque del sistema.

3.3.3. Indicadores de Calidad del Analizador: Precisión, Exhaustividad, Falsos Positivos y Negativos

Conjunto de métricas estadísticas utilizadas para medir el desempeño de un sistema SAST:

Falso positivo (FP). alerta reportada por el analizador sobre un fragmento de código que en realidad es seguro o está correctamente sanitizado.

Falso negativo (FN). vulnerabilidad real presente en el código fuente que el analizador omite detectar.

Precisión (Precision): proporción de vulnerabilidades reales detectadas sobre el total de alertas emitidas por la herramienta:

$$Precision = \frac{TP}{TP + FP}$$

Exhaustividad (Recall): proporción de vulnerabilidades reales detectadas sobre el total de vulnerabilidades existentes en el programa:

$$Recall = \frac{TP}{TP + FN}$$

3.3.4. Accesibilidad (Reachability) en Grafos Dirigidos

Propiedad matemática de la teoría de grafos que determina si existe una secuencia de aristas dirigidas que conecte un nodo de origen u con un nodo de destino v . En el contexto del análisis estático, se utiliza para verificar formalmente si existe flujo de información entre dos puntos del programa.

3.3.5. Sensibilidad en el Análisis de Programas (Flow-, Context- y Path-Sensitivity)

Clasificación teórica que define el grado de precisión y el nivel de abstracción adoptado por un algoritmo de análisis estático (Nielson et al., 1999):

Sensibilidad al Flujo (Flow-sensitivity): un análisis es sensible al flujo si tiene en cuenta el orden de ejecución de las sentencias del programa. Un análisis insensible al flujo considera las sentencias como un conjunto no ordenado.

Sensibilidad al Contexto (Context-sensitivity): un análisis es sensible al contexto si distingue entre las diferentes invocaciones de una misma función según la pila de llamadas (call stack) desde donde fue emitida, evitando mezclar los flujos de datos de distintas llamadas.

Sensibilidad al Camino (Path-sensitivity): un análisis es sensible al camino si evalúa la validez de los predicados o condiciones lógicas de las ramificaciones (if, switch), diferenciando rutas de ejecución factibles de aquellas que son inalcanzables.

4.1. Arquitectura del sistema

El sistema se diseñó como un pipeline por etapas sucesivas y desacopladas. Esta estructura modular permite que cada componente cumpla un rol único dentro del procesamiento: el análisis sintáctico genera el AST, los constructores crean los nodos y aristas de la representación intermedia, y el motor de taint opera sobre el grafo final. La división simplifica además la estrategia de pruebas, ya que permite aislar y validar cada fase mediante pruebas unitarias independientes sin necesidad de ejecutar el pipeline completo para verificar un módulo interno.

La decisión se refleja además en la estrategia de pruebas. Al ejecutar la suite de tests, cada etapa se valida de forma independiente, lo que permite identificar con precisión en qué componente del pipeline se produce una falla. Una arquitectura monolítica impediría este nivel de aislamiento, dado que mezclaría el análisis sintáctico con el análisis semántico y forzaría a probar el sistema como una unidad indivisible.

4.1.1. Organización en paquetes

El proyecto se organiza como un monorepo gestionado mediante espacios de trabajo de npm, compuesto por dos paquetes con responsabilidades diferenciadas. El paquete de núcleo concentra la totalidad del motor de análisis: análisis sintáctico, construcción de la representación intermedia, generación del grafo y detección de recorridos vulnerables. El paquete de visualización implementa la interfaz web y consume los resultados producidos por el núcleo, que expone además un punto de entrada sin dependencias de Node.js para ejecutar el mismo motor en el navegador. Esta separación permite que el motor de análisis opere de manera independiente de cualquier interfaz gráfica.

4.1.2. Flujo de procesamiento

El procesamiento se inicia con los archivos fuente que el usuario selecciona en la versión web o indica en la interfaz de línea de comandos. Cada archivo es transformado en un árbol de sintaxis abstracta. A partir de dicho árbol se construye una representación intermedia compuesta por nodos tipados. Sobre esa representación operan tres constructores de aristas que se ejecutan de manera independiente sobre el mismo conjunto de nodos: el generador del grafo de llamadas, el analizador de flujo de datos intra-procedural y el analizador de flujo inter-procedural. La unión de los nodos y de las aristas producidas por los tres constructores conforma el grafo dirigido que constituye la representación central

del programa. A continuación, la etapa de enlace entre archivos resuelve las llamadas a funciones importadas desde otros archivos del proyecto y agrega las aristas de invocación y de asociación de argumentos correspondientes, de modo que los grafos de los archivos conectados por llamadas se unen en un único grafo. Sobre dicho grafo opera el motor de análisis de contaminación, cuyos resultados se presentan en el módulo de visualización o en los reportes de la interfaz de línea de comandos.

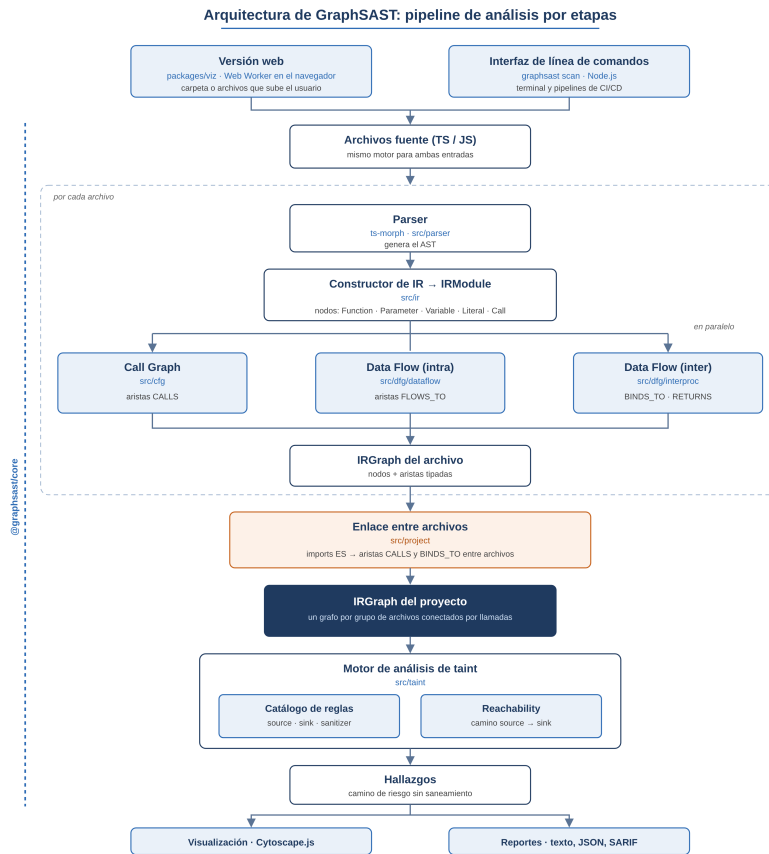


Figura 4.1: Arquitectura de GraphSAST: pipeline de análisis por etapas, desde los archivos del proyecto hasta la visualización y los reportes, con enlace entre archivos

4.2. Prototipo y alcance de lo construido

4.2.1. Estado del desarrollo

El sistema se encuentra actualmente en estado de prototipo funcional. Se ejecuta como aplicación web en el navegador y procesa de forma satisfactoria los casos de prueba, aunque su alcance se define como prototipo y no como producto mínimo viable, ya que aún no fue evaluado por usuarios externos y ciertas características necesarias para su comercialización, como la gestión de suscripciones, forman parte de etapas futuras.

4.2.2. Funcionalidades implementadas

El prototipo ejecuta el pipeline completo de análisis sobre los archivos de un proyecto: análisis sintáctico, construcción de la representación intermedia, generación del grafo de llamadas, análisis de flujo de datos intra e inter-procedural, y detección de recorridos

desde puntos de entrada hacia operaciones sensibles sin saneamiento intermedio. La detección se resuelve mediante dos motores intercambiables: un recorrido en anchura sobre el grafo en memoria, empleado por defecto, y una consulta declarativa en Cypher ejecutada sobre el grafo persistido en Neo4j, que descarta los caminos que atraviesan un nodo de saneamiento y acota la profundidad de búsqueda. Los resultados se presentan mediante visualización interactiva del grafo, con resaltado del camino de riesgo y vinculación con las líneas de código correspondientes. El análisis sigue el dato de un archivo a otro cuando una función importada desde otro archivo del proyecto lo recibe como argumento. La versión web analiza una carpeta o un conjunto de archivos elegidos por el usuario, y la interfaz de línea de comandos analiza rutas del disco, emite reportes en texto, JSON y SARIF, y devuelve códigos de salida que permiten detener un pipeline de integración continua cuando se detectan hallazgos.

4.2.3. Catálogo de detección

Se implementaron las tres debilidades definidas en el diseño arquitectónico, seleccionadas por constituir la tríada canónica del análisis de contaminación y por integrar el listado de debilidades más críticas publicado por MITRE, a las que se sumó la inyección NoSQL (CWE-943), habitual en aplicaciones Node.js que emplean bases de datos documentales como MongoDB:

Tabla 4.1: Debilidades cubiertas por el catálogo de detección implementado

Debilidad	Descripción	Sinks	Sanitizers
CWE-89	Inyección SQL	5	5
CWE-78	Inyección de comandos del sistema operativo	6	4
CWE-79	Cross-Site Scripting (XSS)	6	5
CWE-943	Inyección NoSQL	6	4

Las operaciones sensibles registradas corresponden a llamadas habituales del ecosistema analizado: controladores de bases de datos relacionales, mapeadores objeto-documento, ejecución de procesos del sistema operativo y manipulación del modelo de objetos del documento. Las operaciones de saneamiento reconocidas comprenden tanto funciones genéricas de validación y escapado como bibliotecas específicas del dominio.

4.2.4. Verificación y resultados

El desarrollo cuenta con una suite de pruebas automatizadas organizadas según la etapa del pipeline que verifican, lo que permite validar cada componente de forma aislada. La aplicación incorpora asimismo dos casos de demostración que difieren en una sola línea: un fragmento en el que el identificador recibido en la URL cruza de una función a otra y llega a una consulta a base de datos, y el mismo fragmento con una operación de saneamiento intermedia. Los demás escenarios, como los flujos dentro de una misma función, las operaciones sensibles alcanzadas por literales, las entradas de Express, la inyección de comandos y el XSS, se cubren en el banco de pruebas descrito más adelante.

La figura siguiente presenta el análisis de un caso vulnerable. El dato ingresa por el parámetro de una función, se asigna a una variable intermedia, se transfiere como

argumento a otra función y alcanza una operación de consulta a base de datos sin haber atravesado ninguna operación de saneamiento. El sistema identifica el recorrido y lo resalta sobre el grafo, indicando además las líneas de código involucradas.

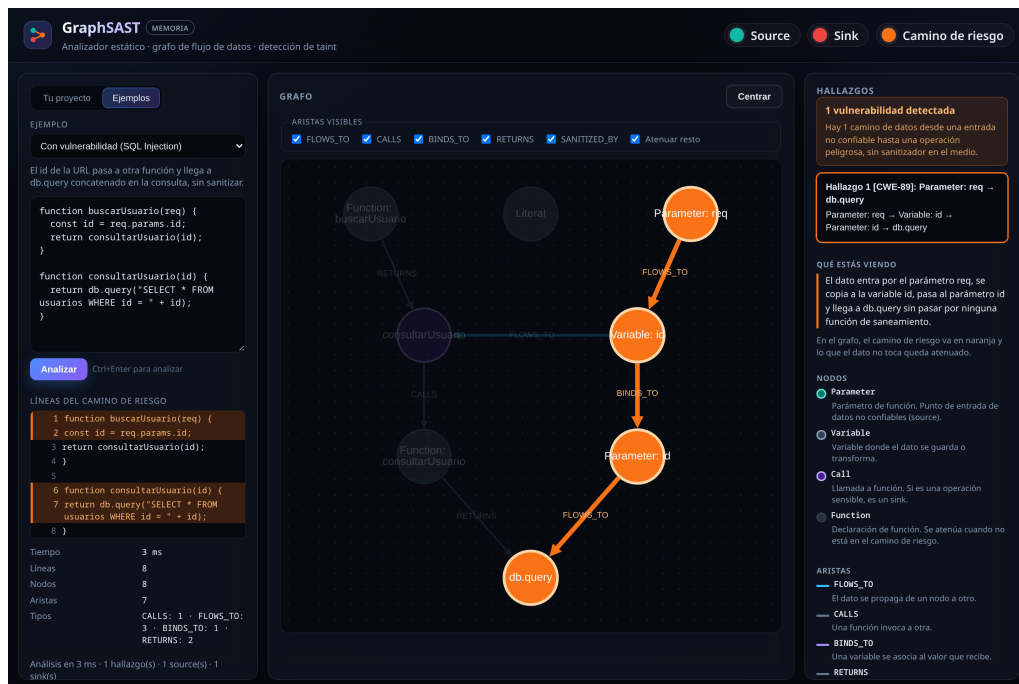


Figura 4.2: Detección de vulnerabilidad (CWE-89): el dato fluye desde el parámetro de entrada hasta la operación de consulta sin saneamiento intermedio. El camino de riesgo se resalta en la visualización

El comportamiento ante un dato correctamente validado resulta particularmente relevante para las hipótesis del trabajo. En el caso siguiente, el mismo patrón de flujo incorpora una operación de saneamiento intermedia. El sistema reconoce la presencia de un punto de entrada y de una operación sensible —los identifica y los diferencia cromáticamente— pero descarta el hallazgo al verificar el saneamiento en el recorrido. Una herramienta basada en coincidencia de patrones señalaría la operación como vulnerable, dado que carece de la capacidad de determinar si el dato fue procesado en el trayecto.

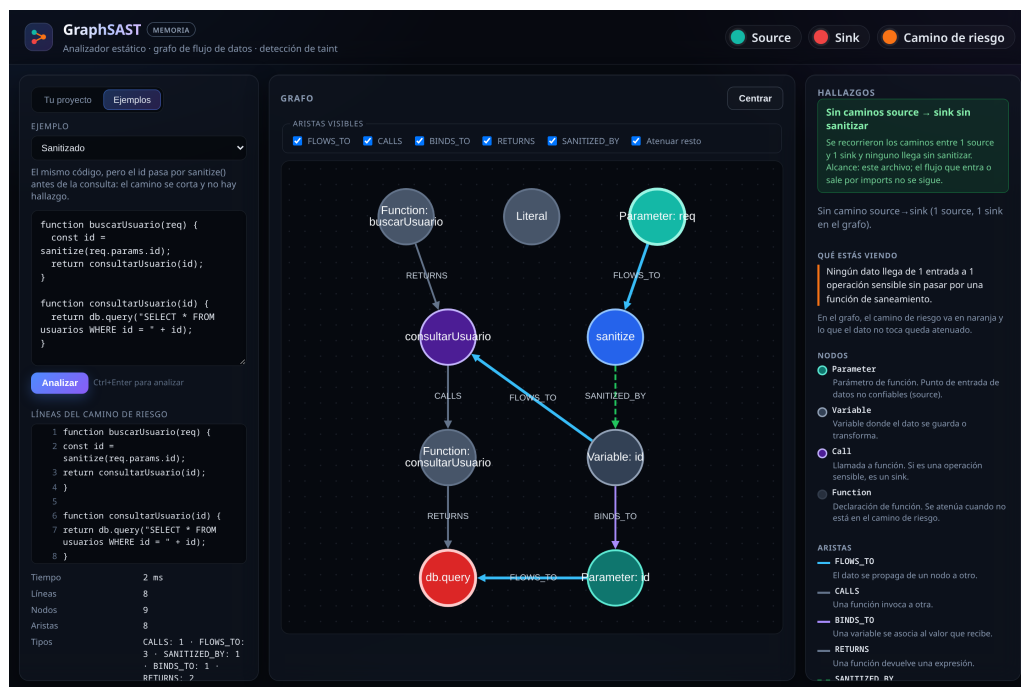


Figura 4.3: Caso saneado: el mismo patrón de flujo con una operación de saneamiento intermedia. El sistema identifica el origen y el destino sensible, pero descarta el hallazgo al constatar el saneamiento en el recorrido

La interfaz expone asimismo las métricas correspondientes a cada ejecución, que incluyen el tiempo de análisis, la cantidad de líneas procesadas, y el número de nodos y aristas generados en el grafo, discriminados por tipo de relación. Los tiempos registrados sobre los casos de prueba se ubican en el orden de las decenas de milisegundos.

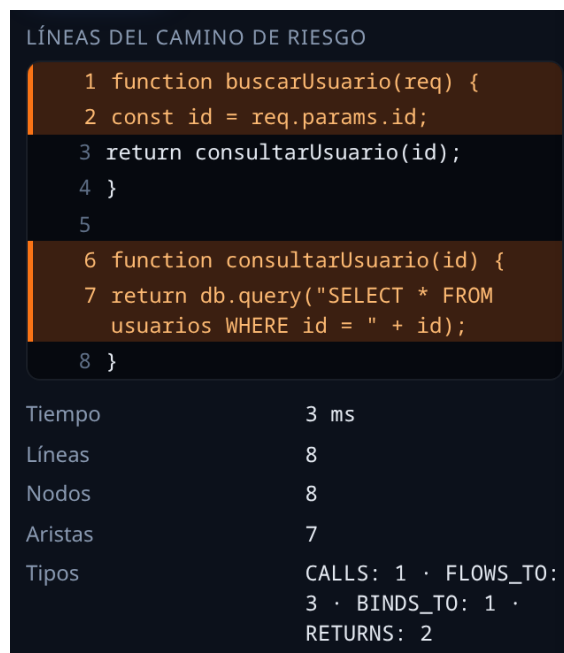


Figura 4.4: Panel de métricas y líneas del camino de riesgo correspondientes al análisis ejecutado

La evaluación cuantitativa se ejecuta sobre un banco de pruebas etiquetado y repro-

ducible desde la línea de comandos, integrado por cuarenta y tres casos de verdad conocida (veintinueve vulnerables y catorce seguros) que cubren recorridos intra e interprocedurales, casos saneados, operaciones sensibles alcanzadas por literales constantes y entradas propias de Express. Además de las tres debilidades del catálogo, el banco incorpora casos de validación insuficiente de la entrada (CWE-20) que alcanzan operaciones sensibles ya catalogadas. Cada caso se clasifica de forma binaria según el sistema emita o no hallazgos, lo que permite construir la matriz de confusión y derivar los indicadores definidos en el Capítulo 2.

Tabla 4.2: Resultados del banco de pruebas sintético (43 casos; tiempos: mediana de cinco ejecuciones)

Indicador	Valor obtenido
Verdaderos positivos (TP)	29
Falsos positivos (FP)	0
Verdaderos negativos (TN)	14
Falsos negativos (FN)	0
Precisión	1,00
Exhaustividad (Recall)	1,00
F1	1,00
Tiempo total de análisis	52 ms
Tiempo por línea de código	0,27 ms

Corresponde delimitar el alcance de estos valores. Se obtienen sobre un corpus sintético reducido, construido por el propio autor y acotado a las construcciones sintácticas que el analizador cubre, por lo que no constituyen evidencia de desempeño sobre proyectos reales ni admiten generalización. Su función en esta instancia es la de una prueba de regresión que verifica que el motor no emite alertas sobre los casos correctamente saneados ni omite los recorridos vulnerables conocidos. La comparación cuantitativa con una herramienta de referencia, prevista en el diseño de investigación del Capítulo 2, permanece pendiente. La validación frente a vulnerabilidades documentadas se presenta a continuación.

Para contrastar el sistema con vulnerabilidades reales se empleó un corpus de seis advisories del ecosistema npm publicados en la base de datos de advisories de GitHub, relevados el 1 de septiembre de 2026 y congelados para que la medición resulte reproducible, restringidos a las debilidades del catálogo. Para cada caso se descargan del registro de npm la versión vulnerable del paquete y la versión que corrige la falla, se analizan ambas contando solo los hallazgos de la debilidad del advisory y se registran dos resultados: si la versión vulnerable presenta hallazgos (detección) y si la corrección del mantenedor reduce su cantidad (discriminación). La discriminación constituye la evidencia fuerte, dado que indica que el hallazgo se vincula con la vulnerabilidad y no con un patrón presente en todo el paquete.

Tabla 4.3: Validación sobre vulnerabilidades documentadas del ecosistema npm (6 casos)

Paquete (vulnerable → corregida)	CWE	Advisory	Archivos	Hallazgos (vuln. → corr.)	Resultado
@hypequery/clickhouse 2.5.0 → 2.5.1	89	GHSA-6wcc-39rp-hh9p	0	—	Sin código fuente publicado
@anthropic-ai/claude-code 2.1.162 → 2.1.163	78	GHSA-7835-87q9-rgvv	3	0 → 0	No detectado
sanitize-html 2.17.4 → 2.17.5	79	GHSA-vccv-cmxp-4j9h	1	0 → 0	No detectado
sanitize-html 2.17.6 → 2.17.7	79	GHSA-g8qq-57p8-ggw5	1	0 → 0	No detectado
defuddle 0.19.0 → 0.19.1	79	GHSA-jg4p-g6xj-4qmf	0	—	Sin código fuente publicado
dompurify 3.4.12 → 3.4.13	79	GHSA-55q2-fjhq-7xh7	7	1 → 1	Detectado, sin discriminación

El sistema detectó uno de los seis casos (16,7 %) y ninguno con discriminación: el único hallazgo, en DOMPurify, se mantiene en la versión corregida, por lo que no corresponde a la vulnerabilidad reportada. El análisis de los casos explica el resultado. En dos paquetes, @hypequery/clickhouse y defuddle, el registro de npm solo contiene código compilado en la carpeta dist, que el evaluador excluye por tratarse de código generado; como verificación complementaria se analizó también esa carpeta, sin cambios en la conclusión: ningún hallazgo en el primero y tres hallazgos idénticos en ambas versiones del segundo. El paquete de @anthropic-ai/claude-code publica únicamente un envoltorio de instalación. En cuanto a la naturaleza de las fallas, las de sanitize-html y DOMPurify residen en la lógica interna de esas bibliotecas de saneamiento, como la validación incompleta de esquemas de URI o la eliminación de nodos del documento, y consisten en omisiones de sus propias reglas de filtrado; la de @hypequery/clickhouse es un error en su función de escapado de parámetros; la de defuddle consiste en interpolar atributos sin escapar al construir HTML como texto, una operación que el catálogo no registra como destino sensible; y la de inyección de comandos se origina en una confusión de rutas que habilita el escape del entorno aislado. Ninguna adopta la forma de un recorrido desde una entrada no confiable hasta una operación sensible del catálogo, que es la clase de vulnerabilidad que el modelo de accesibilidad detecta.

En consecuencia, esta validación no permite afirmar la efectividad del sistema sobre código real, y tampoco la refuta: el corpus se compone de bibliotecas cuyas fallas quedan fuera del modelo de amenaza del analizador. La validación pendiente debe orientarse a aplicaciones que reciben entradas de usuario y las conducen hacia bases de datos, procesos del sistema o el documento, que es el escenario para el que la herramienta fue diseñada, y ampliar la cantidad de casos hasta admitir conclusiones cuantitativas.

4.2.5. Limitaciones del alcance actual

Alcance del análisis entre archivos. El dato se sigue de un archivo a otro cuando un archivo invoca una función declarada con `function` en otro archivo del proyecto e importada mediante `import` de módulos ES con ruta relativa. No se siguen las importaciones mediante `require`, las funciones flecha o asignadas a constantes, los métodos de clase, los alias de rutas de TypeScript ni los paquetes de terceros; en esos casos cada archivo se analiza por separado. Por la misma razón, las aplicaciones organizadas en clases o con funciones flecha, habituales en Express y NestJS, quedan cubiertas de forma parcial.

Motor de consultas por defecto. El sistema implementa los dos motores de resolución previstos en el diseño arquitectónico: el recorrido en memoria y la consulta declarativa en Cypher sobre el grafo persistido en Neo4j. El motor sobre base de datos opera, no obstante, de manera opcional, dado que requiere una instancia activa y la configuración de la variable de entorno correspondiente; en ausencia de dicha instancia la aplicación recurre al recorrido en memoria, que constituye el modo de operación por defecto. La paridad entre ambos motores se verificó sobre casos sintéticos y no sobre proyectos de escala.

Validación con usuarios. No se realizaron pruebas de usabilidad con usuarios externos, instancia requerida para la validación de la hipótesis relativa a la utilidad de la visualización.

4.2.6. Próximos pasos

Se prevé la consolidación del motor sobre base de datos de grafos como modo de operación habitual, la ampliación del catálogo de reglas y de los patrones de llamada que resuelve el análisis entre archivos, y la integración de la interfaz de línea de comandos en pipelines de integración continua como complemento de la versión web. Esta modalidad resulta coherente con el enfoque de incorporación temprana de controles de seguridad que fundamenta el proyecto: el análisis puede ejecutarse de forma automática sobre cada cambio del código, y en ambas modalidades el código fuente no abandona la infraestructura del usuario.

4.3. Stack de software

La selección de tecnologías respondió a criterios explícitos de viabilidad, adecuación al dominio y coherencia interna del stack. La tabla siguiente resume las herramientas empleadas y la capa en la que interviene cada una.

Tabla 4.4: Tecnologías empleadas en el desarrollo del prototipo

Tecnología	Versión	Capa	Función
TypeScript	5.5	Transversal	Lenguaje del motor y de la interfaz
Node.js	—	Entorno de ejecución	Ejecución del motor de análisis
Web Workers	—	Ejecución	Análisis en segundo plano en el navegador
ts-morph	23.0	Análisis sintáctico	Generación y recorrido del AST
Cytoscape.js	3.30	Visualización	Renderizado interactivo del grafo
Vite	5.4	Construcción	Empaquetado y servidor de desarrollo
Vitest	2.0	Pruebas	Ejecución de la suite automatizada
npm workspaces	—	Estructura	Gestión del monorepo
Neo4j	5 Community	Persistencia	Almacenamiento del grafo compilado
neo4j-driver	6.2	Persistencia	Cliente Bolt y ejecución de consultas Cypher
Docker Compose	—	Entorno	Provisión local de la instancia de Neo4j
Vercel	—	Despliegue	Publicación de la aplicación web como sitio estático

4.3.1. Estrategia de análisis sintáctico

Se decidió utilizar la biblioteca `ts-morph` en lugar de programar un analizador sintáctico propio. Desarrollar un parser completo para la gramática de TypeScript y sus continuas actualizaciones habría consumido gran parte del tiempo del proyecto sin aportar valor diferencial. El aporte de GraphSAST está en los algoritmos de construcción del grafo de flujo de datos, el análisis de taint y la visualización del riesgo.

`ts-morph` simplifica la interacción con el compilador oficial de TypeScript y expone una API clara para recorrer los nodos del AST, lo que reduce de manera considerable el código necesario para navegar la estructura del programa.

4.3.2. Lenguaje del motor

El motor se implementó en TypeScript sobre Node.js. La elección responde a tres razones: permite emplear un único lenguaje a lo largo de todo el sistema, incluida la interfaz de visualización; el sistema de tipos resulta adecuado para modelar las estructuras del grafo, en las que la distinción entre categorías de nodos y de aristas es central; y el ecosistema ofrece acceso directo a las herramientas de análisis del lenguaje objetivo. Se descartó el uso de Python, dado que el tratamiento del árbol sintáctico de TypeScript resulta menos natural fuera de su propio ecosistema, así como una solución híbrida que combinara ambos entornos, por la complejidad operativa que introduciría.

4.3.3. Visualización

La representación gráfica se implementó con Cytoscape.js, una biblioteca JavaScript orientada específicamente a la construcción y visualización interactiva de grafos en el navegador. La selección se fundamentó en un relevamiento de herramientas del área y en la consulta con desarrolladores con experiencia previa en la materia. Se consideraron asimismo Gephi e igraph, descartadas por no resultar integrables en una aplicación web: la primera constituye una aplicación de escritorio orientada a la exploración manual de grafos, y la segunda una biblioteca de análisis destinada al cómputo de métricas antes que al renderizado interactivo. Cytoscape.js provee de manera nativa los algoritmos de disposición y el modelo de nodos y aristas requeridos por el proyecto, y cuenta con adopción consolidada en contextos de investigación y análisis de grafos.

4.3.4. Herramientas de construcción y pruebas

El módulo de visualización emplea Vite como empaquetador y servidor de desarrollo. La elección respondió a la experiencia previa con la herramienta en proyectos de características similares, criterio que reduce la curva de aprendizaje y el riesgo de configuración en un desarrollo acotado en el tiempo. Vite ofrece además soporte nativo para TypeScript y módulos ES, lo que resulta coherente con el resto del stack. Las pruebas automatizadas se ejecutan con Vitest, seleccionado por su integración directa con Vite y por compartir su configuración, evitando duplicar la infraestructura de construcción del proyecto.

4.4. Infraestructura y modelo de ejecución

El sistema se distribuye como una aplicación web estática: el servidor entrega la página y el análisis se ejecuta en el navegador del usuario. No se estructura como un servicio con procesamiento en el servidor, y esta característica condiciona los requerimientos de infraestructura, que difieren de los correspondientes a un servicio alojado convencional.

4.4.1. Modelo de ejecución

El usuario accede a la aplicación desde el navegador y selecciona la carpeta o los archivos de su proyecto. El motor de análisis, el mismo que utiliza la interfaz de línea de comandos, se ejecuta en un proceso en segundo plano del navegador (Web Worker), de modo que la interfaz permanece operativa durante el análisis. Los archivos se leen y se analizan en el equipo del usuario: el servidor de alojamiento solo entrega los archivos estáticos de la aplicación y no recibe el código analizado. Esta propiedad resulta pertinente al dominio del proyecto: dado que la herramienta procesa código fuente potencialmente sensible, el hecho de que dicho código no abandone el entorno del usuario constituye una garantía relevante. La interfaz de línea de comandos, que ejecuta el mismo motor sobre Node.js, se mantiene para su uso en terminal y en pipelines de integración continua.

4.4.2. Requerimientos

La versión web requiere únicamente un navegador actualizado; la interfaz de línea de comandos requiere un entorno con Node.js. No se requiere hardware especializado, servi-

cios de terceros ni infraestructura de servidores. El desarrollo y las pruebas se realizaron sobre un equipo de escritorio de características estándar.

4.4.3. Disponibilidad y escalabilidad

Al ejecutarse el análisis en el navegador de cada usuario, el servidor no realiza procesamiento y la carga de cada análisis recae en el equipo que lo solicita, por lo que la disponibilidad y la escalabilidad se reducen a la entrega de archivos estáticos. El factor determinante del rendimiento es el tamaño del código analizado, dado que el costo computacional del análisis crece con la cantidad de nodos y aristas del grafo generado. Como referencia, en el equipo de desarrollo el análisis en el navegador de un proyecto de 78 archivos y unas 8.500 líneas demandó alrededor de 2,8 segundos. La caracterización de dicho crecimiento forma parte de las mediciones previstas en la validación del sistema.

4.5. Modelo de datos: el grafo del programa

El sistema representa el programa analizado mediante un grafo dirigido en el que los nodos corresponden a los elementos por los que circula la información y las aristas a las relaciones que se establecen entre ellos. Este modelo constituye el núcleo del aporte técnico del trabajo, dado que sobre él se resuelve la detección de vulnerabilidades.

4.5.1. Nodos

Se definen cinco categorías de nodo. Cada nodo almacena su ubicación exacta en el código fuente —archivo, línea y columna— junto con el fragmento de texto correspondiente, lo que permite vincular cualquier elemento del grafo con su posición original en el programa.

Tabla 4.5: Categorías de nodo del modelo de grafo

Categoría	Elemento representado
Function	Declaración de función
Parameter	Parámetro de entrada de una función
Variable	Variable declarada
Literal	Valor literal presente en el código
Call	Invocación a una función u operación

4.5.2. Aristas

Se definen cinco categorías de relación dirigida. La distinción entre categorías resulta determinante para el funcionamiento del sistema, dado que permite diferenciar la propagación efectiva de un valor de otras relaciones estructurales del programa.

Tabla 4.6: Categorías de relación del modelo de grafo

Relación	Significado
CALLS	Una función invoca a otra
FLOWS_TO	Un valor se propaga de un nodo a otro
BINDS_TO	Una variable se asocia al valor que recibe
RETURNS	Una función devuelve una expresión
SANITIZED_BY	Un valor atraviesa una operación de saneamiento

4.5.3. Resolución de la detección sobre el modelo

La identificación de una vulnerabilidad se resuelve como un problema de accesibilidad sobre el grafo: se determina si existe un camino que conecte un nodo de origen con un nodo de destino sensible sin que dicho recorrido atraviere una relación de saneamiento. Esta formulación traduce el planteo conceptual del trabajo —la seguridad entendida como accesibilidad sobre un grafo dirigido— en una operación computable sobre una estructura de datos concreta.

La relación de saneamiento cumple un rol central para evitar falsas alarmas: si una variable fue validada en el trayecto, el sistema descarta el camino. Un analizador basado en coincidencia de patrones carece de esa información y debe reportar el caso como potencialmente vulnerable.

Durante la implementación de las consultas Cypher sobre Neo4j surgió una dificultad concreta con las llamadas recursivas o cíclicas entre funciones, que generaban recorridos de longitud indefinida y demoraban el motor. Para resolverlo se acotó la profundidad de búsqueda a quince saltos entre el origen y el destino. En el recorrido en memoria ese límite es el valor por defecto de un parámetro configurable, mientras que en la consulta Cypher queda fijado en el patrón de camino. La restricción evita los recorridos indefinidos sin comprometer la detección en los flujos habituales de una aplicación web: sobre el banco de pruebas de 43 casos, el análisis completo demandó alrededor de 52 ms, aproximadamente 0,27 ms por línea de código (tabla 4.2).

4.6. Diseño e interacción

4.6.1. Usuario destinatario

La herramienta está orientada a desarrolladores y analistas que requieren auditar código fuente. El diseño asume familiaridad técnica con el lenguaje analizado y con los conceptos de seguridad involucrados, por lo que privilegia la densidad informativa sobre la simplificación de la interfaz.

4.6.2. Organización de la interfaz

La aplicación se estructura en tres paneles simultáneos. El panel izquierdo ofrece dos modos. En el modo de proyecto, el usuario elige o arrastra una carpeta o un conjunto de archivos; la aplicación informa cuántos archivos se analizan y por qué se excluyen los demás, muestra el progreso del análisis y, al finalizar, lista los archivos con su cantidad de hallazgos junto con el contenido del archivo elegido y las líneas del camino de riesgo

resaltadas (figura 4.5). En el modo de ejemplos, el usuario puede ingresar un fragmento mediante un editor de texto o seleccionar uno de los dos ejemplos precargados: un fragmento con una inyección SQL y el mismo fragmento con una operación de saneamiento. El panel central presenta el grafo generado. El panel de hallazgos expone el veredicto, cada hallazgo con su debilidad, su ubicación y su recorrido, el catálogo de debilidades cubiertas y las reglas activas. La disposición simultánea responde a un criterio explícito: permitir que el usuario contraste el código, su representación como grafo y los hallazgos sin necesidad de alternar entre vistas.

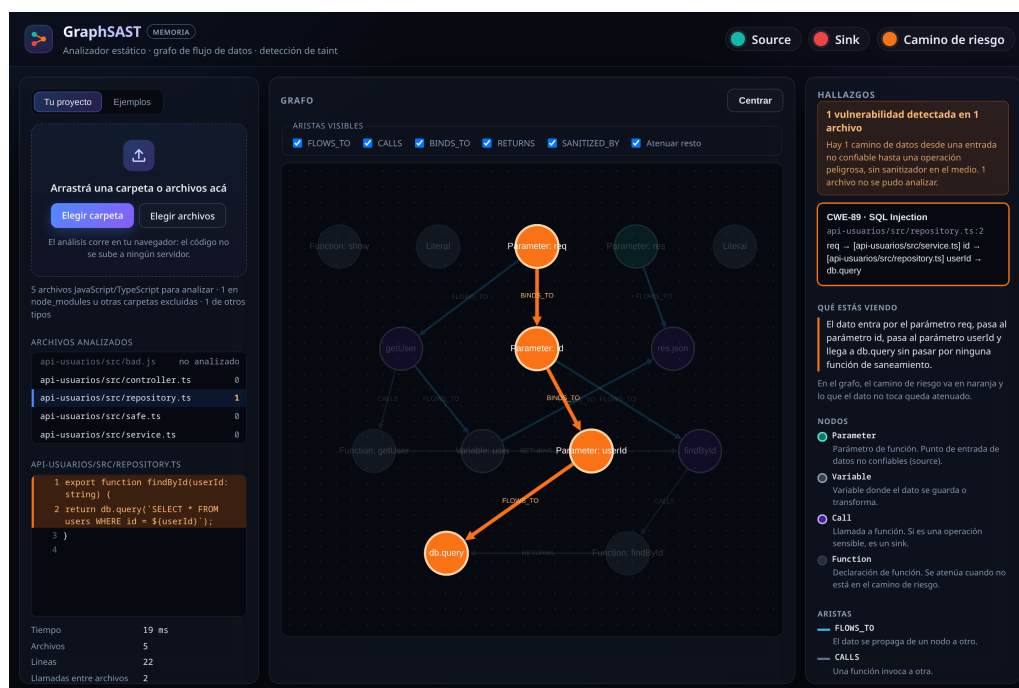


Figura 4.5: Versión web: análisis de un proyecto subido como carpeta. El hallazgo cruza tres archivos y se muestra el archivo del sink con las líneas del camino resaltadas

4.6.3. Decisiones de diseño

Se incorporó una codificación cromática con leyenda visible que distingue los nodos de origen, los nodos correspondientes a operaciones sensibles y el camino de riesgo identificado. Esta codificación permite interpretar el resultado de un análisis sin recurrir al texto: el contraste entre las figuras 4.2 y 4.3 ilustra la diferencia entre un recorrido vulnerable y uno saneado a partir del color de los nodos y de las aristas.

El grafo admite el filtrado independiente de cada categoría de arista, junto con una opción de atenuación del resto de la estructura. Esta decisión atiende a un problema concreto de legibilidad: la superposición de relaciones de distinta naturaleza dificulta la lectura del recorrido relevante, por lo que el control sobre qué relaciones se muestran resulta necesario para interpretar grafos de cierto tamaño.

La interfaz señala además de manera explícita el modo de operación del motor mediante un indicador en el encabezado. En la versión web el análisis se resuelve siempre mediante el recorrido en memoria, dado que se ejecuta en el navegador; la consulta Cypher sobre el grafo persistido en Neo4j queda disponible a través de la API local. Esta decisión responde al criterio de que las condiciones de ejecución y las limitaciones del prototipo resulten

visibles para el usuario.

4.6.4. Vinculación con el código fuente

La interfaz presenta las líneas de código correspondientes al camino de riesgo detectado, y la selección de un nodo del grafo despliega su información asociada, incluida su ubicación en el archivo. Esta correspondencia entre la representación abstracta y el código concreto constituye el aspecto central del diseño, dado que la utilidad de la visualización depende de que el usuario pueda trasladar el hallazgo a una acción sobre el código.

4.6.5. Accesibilidad y validación

Se incorporaron etiquetas descriptivas en los controles de la interfaz y regiones de actualización dinámica anunciadas por lectores de pantalla, así como un atajo de teclado para ejecutar el análisis. No se realizaron pruebas de usabilidad con usuarios externos. La validación de la hipótesis relativa a la utilidad de la visualización requiere dicha instancia, prevista como trabajo posterior mediante sesiones de evaluación con desarrolladores.

4.7. Calidad, seguridad y despliegue

4.7.1. Estrategia de pruebas

El proyecto cuenta con una suite de pruebas automatizadas ejecutadas mediante Vitest. Las pruebas se organizan siguiendo la estructura por etapas del sistema, verificando de forma independiente el análisis sintáctico, la construcción de la representación intermedia, el grafo de llamadas, el análisis de flujo de datos, el motor de contaminación, la carga del catálogo de reglas, la generación de sentencias Cypher, la selección de motor, la construcción de informes, el análisis entre archivos y la carga de archivos de la versión web, además del módulo de visualización. La suite comprende doscientas cuarenta y siete pruebas distribuidas en veintisiete archivos. Esta organización deriva directamente de la decisión arquitectónica adoptada: la separación en etapas permite verificar cada componente de manera aislada y localizar con precisión el origen de una falla.

De modo complementario a las pruebas unitarias, la aplicación incorpora casos de demostración que reproducen patrones habituales de código y verifican el comportamiento del sistema tanto ante recorridos vulnerables como ante recorridos correctamente saneados.

4.7.2. Control de versiones

El desarrollo se gestiona mediante Git, con repositorio alojado en GitHub: <https://github.com/MaxiZk/graphsast>. Los cambios se incorporan una vez verificado que la suite de pruebas se ejecuta satisfactoriamente y que la aplicación opera conforme a lo esperado, criterio que mantiene el repositorio en un estado funcional a lo largo del desarrollo.

4.7.3. Consideraciones de seguridad

El sistema no administra cuentas de usuario, credenciales ni bases de datos remotas, por lo que no requiere esquemas de autenticación ni cifrado de datos en reposo. Una decisión

clave del modelo de ejecución es que el código fuente analizado nunca se transmite fuera de la computadora del usuario: aunque la aplicación se sirve desde un servidor, el análisis se realiza en el navegador. Tratándose de código que puede contener vulnerabilidades o lógica propietaria, esa ejecución local elimina el riesgo de filtración hacia servidores externos.

4.7.4. Despliegue

La aplicación web se publica como sitio estático en la plataforma Vercel, en <https://graph-sast.vercel.app/>. Vercel compila el proyecto a partir del repositorio y sirve los archivos resultantes sin funciones de servidor; se verificó sobre el sitio publicado que el análisis no genera solicitudes hacia otros dominios, de modo que el código analizado no sale del navegador. Para el desarrollo local, la ejecución requiere la instalación de las dependencias mediante el gestor de paquetes y la puesta en marcha del servidor de desarrollo.

5.1. Escenario de comercialización

El proyecto se evalúa bajo un modelo de suscripción, distribuido como aplicación web en la que el análisis se ejecuta en el navegador del usuario, complementada por una herramienta de línea de comandos (CLI) integrable en pipelines de integración y despliegue continuos. El abono mensual se fijó en USD 8 por usuario, equivalente a USD 96 anuales bajo facturación anualizada.

5.1.1. Suscripción frente a licencia perpetua

La elección de la suscripción responde a dos motivos, uno financiero y otro técnico:

Reurrencia frente a esfuerzo continuo de reposición. La suscripción genera un flujo de fondos predecible, en el que los usuarios captados continúan aportando en los ejercicios posteriores mientras utilicen la herramienta. Un esquema de licencia única obligaría a captar un volumen equivalente de clientes nuevos cada año solo para sostener el nivel de ingresos. A ello se suma que una herramienta de análisis estático demanda actualización periódica de reglas para cubrir nuevos patrones de vulnerabilidad y versiones de librerías, tarea de mantenimiento que se financia de manera natural con ingresos periódicos.

Inconsistencia de precios relativos. A USD 8 por mes, un cliente iguala en cinco meses el valor típico de una licencia perpetua individual, estimada en torno a USD 50. Ofrecer ambas modalidades en paralelo a esos valores generaría arbitraje en contra de la suscripción, o exigiría fijar la licencia perpetua a un precio fuera de mercado.

5.1.2. Público objetivo

El producto se orienta a tres segmentos:

Desarrolladores independientes. Programadores de JavaScript y TypeScript, en entornos Node.js, Express y React, que necesitan auditar código fuente localmente para prevenir inyecciones SQL (CWE-89), inyecciones de comandos del sistema operativo (CWE-78) y Cross-Site Scripting (CWE-79) antes de enviar cambios al repositorio.

Equipos de desarrollo en PyMEs y empresas de reciente creación. Grupos de tres a quince desarrolladores que buscan implementar prácticas de seguridad temprana sin la carga presupuestaria ni la infraestructura que requieren las suites corporativas.

Consultores de aseguramiento de calidad y seguridad de aplicaciones. Profesionales que realizan auditorías puntuales sobre código de terceros.

5.1.3. Relación entre público y precio

Las soluciones corporativas consolidadas del mercado operan con contratos anuales de miles de dólares y requieren servidores dedicados. Frente a ellas, USD 8 mensuales equivalen aproximadamente a una hora de desarrollo junior en Argentina, según la tarifa de referencia adoptada en la sección 5.2. Ese orden de magnitud elimina la barrera de entrada para un profesional independiente o para el responsable técnico de una empresa pequeña, dado que la decisión de compra no requiere aprobaciones presupuestarias complejas.

5.1.4. Dimensionamiento del mercado

Mercado total direccionable. El relevamiento global de SlashData (State of the Developer Nation) y el reporte anual GitHub Octoverse ubican a JavaScript y TypeScript de forma constante a la cabeza del ranking de adopción, con una base mundial superior a los veinte millones de programadores. La distribución como aplicación web, accesible desde el navegador sin instalación, junto con la publicación de la herramienta de línea de comandos en el registro de paquetes npm, canal habitual de las herramientas de desarrollo del ecosistema JavaScript, permite alcanzar esa base sin intermediarios comerciales.

Mercado disponible regional. Para dimensionar el mercado local se parte del dato publicado por el Observatorio Permanente de la Industria del Software y Servicios Informáticos (OPSSI) de CESSI, que registra 162.598 puestos de trabajo formales en el sector al primer trimestre de 2026. Dado que esa cifra comprende la totalidad de las funciones de las empresas —gestión, soporte, infraestructura y administración—, la proporción estrictamente técnica se estima a partir de los relevamientos de la Encuesta de Sueldos de Sysarmy, en los que los roles de desarrollo concentran entre el 50 % y el 60 % del personal técnico. Asumiendo de forma conservadora que entre un 35 % y un 50 % de ese personal de desarrollo trabaja sobre aplicaciones web en el ecosistema JavaScript/TypeScript, el mercado accesible local se ubica en un orden de entre 28.000 y 49.000 desarrolladores.

Mercado objetivo. La proyección contempla alcanzar 20 usuarios en el primer año y escalar gradualmente hasta 300 usuarios activos en el quinto. Sobre el mercado accesible estimado, esos 300 suscriptores representan una cuota de penetración de entre el 0,6 % y el 1,1 %, lo que muestra que la factibilidad del flujo no descansa en metas de captación desmedidas.

5.2. Inversión inicial

La inversión comprende las horas de ingeniería y los desembolsos requeridos para transformar el prototipo funcional en un producto comercializable: la aplicación web con cobro de suscripciones y acceso controlado por licencia.

5.2.1. Supuestos de la estimación de horas

Los parámetros de dedicación correspondientes a la fase de prototipo y a las tareas pendientes hasta el lanzamiento se resumen en la tabla 5.1.

Tabla 5.1: Supuestos de estimación de horas de desarrollo

Parámetro	Valor adoptado
Período de desarrollo del prototipo	Mediados de junio a mediados de septiembre de 2026 (13 semanas)
Días dedicados por semana	3 a 4 días (promedio: 3,5)
Horas diarias de trabajo	4 a 5 horas (promedio: 4,5)
Carga semanal estimada	≈ 15,75 horas
Horas insumidas en el prototipo	≈ 205 horas (rango representativo: 156 a 260)
Horas adicionales hasta producto comercializable	400 horas (cobro de suscripciones y emisión de licencias, control de acceso en la aplicación web, publicación de la herramienta de línea de comandos en npm, documentación y pruebas con usuarios)

5.2.2. Determinación del valor hora

Se adoptó un valor de USD 8 por hora como costo total para el empleador. La cifra se construye en tres pasos. La Encuesta de Sueldos 2026.01 de Sysarmy, analizada por OpenQube, reporta para la categoría Junior en Argentina una mediana salarial bruta de ARS 1.538.500 mensuales; sobre un estándar de 160 horas laborales mensuales, ello equivale a ARS 9.616 por hora de salario bruto. A dicho valor corresponde adicionar las contribuciones patronales y cargas sociales, que en el régimen argentino incrementan el costo laboral en aproximadamente un 25 % respecto del salario bruto, con lo que el costo por hora asciende a ARS 12.020. Convertido al tipo de cambio mayorista de referencia indicado en la sección 5.4, el valor resultante se ubica en el orden de USD 8 por hora.

Se emplea el costo total para el empleador, y no el salario bruto percibido, por tratarse de la magnitud que refleja el verdadero consumo de recursos que implica cada hora de desarrollo.

5.2.3. Composición de la inversión inicial

En la tabla 5.2 se detallan las erogaciones previstas para el Año 0.

Tabla 5.2: Presupuesto de inversión inicial (Año 0)

Concepto	Cálculo	Importe (USD)
Desarrollo del prototipo	205 h × USD 8	1.640
Desarrollo hasta versión comercializable	400 h × USD 8	3.200
Marketing y lanzamiento inicial	Sitio informativo, registro y difusión	500
Licencias de software	Pila tecnológica de código abierto	0
Hardware y equipamiento	Equipo informático propio disponible	0
Infraestructura de servidores	Plan gratuito de Vercel durante el desarrollo, sin uso comercial	0
Total inversión inicial (Año 0)		5.340

5.3. Costos post-proyecto

Los costos operativos anuales necesarios para sostener la herramienta durante el horizonte de cinco años comprenden tres componentes.

El primero corresponde al mantenimiento técnico y al soporte. Se estiman seis horas mensuales destinadas a la atención de incidencias, la corrección de errores y la actualización del catálogo de reglas, valorizadas al costo horario adoptado, lo que arroja un costo anual fijo de USD 576.

El segundo corresponde al alojamiento y al dominio. La aplicación web se publica en Vercel, cuyo plan gratuito se limita al uso personal no comercial; la comercialización exige el plan Pro, con un abono fijo de USD 20 mensuales (USD 240 anuales) que incluye 1 TB mensual de transferencia de datos, volumen holgado para una aplicación estática cuyo análisis se ejecuta en el navegador. La validación de las licencias se resuelve con una función de servidor de consumo marginal, cuyo excedente sobre el uso incluido en el plan se factura a USD 0,60 por millón de invocaciones. La renovación del dominio .com cuesta USD 11,25 anuales en el registrador de Vercel, redondeados a USD 11.

El tercero corresponde a las comisiones de cobro. El cobro de las suscripciones y la emisión de licencias se delegan en Lemon Squeezy, que actúa como comerciante registrado (merchant of record): procesa los pagos, liquida los impuestos al consumo de cada país y emite claves de licencia que vencen junto con la suscripción, por lo que la aplicación no necesita administrar cuentas de usuario propias. Su comisión es del 5 % más USD 0,50 por transacción, sin cargo mensual; Paddle, alternativa equivalente, aplica la misma tarifa. Con facturación anual, cada suscriptor genera una transacción de USD 96 y una comisión de USD 5,30, de modo que este rubro crece con la base de usuarios.

Tabla 5.3: Costos operativos anuales post-proyecto, en dólares (Años 1 a 5)

Rubro	Año 1	Año 2	Año 3	Año 4	Año 5
Mantenimiento y soporte	576	576	576	576	576
Alojamiento (Vercel Pro)	240	240	240	240	240
Dominio	11	11	11	11	11
Comisiones de cobro	106	318	636	1.060	1.590
Costo operativo total anual	933	1.145	1.463	1.887	2.417

5.4. Flujo de fondos proyectado

5.4.1. Supuestos del flujo

Moneda de cuenta. La totalidad del flujo se expresa en dólares estadounidenses, con el objeto de aislar la proyección del efecto de la inflación en moneda local. Como referencia para los costos originados en pesos se tomó el tipo de cambio mayorista de la Comunicación «A» 3500 del Banco Central de la República Argentina, que al 11 de septiembre de 2026 se ubicó en ARS 1.509,91 por dólar.

Ingreso por usuario. Cada suscriptor activo abona USD 96 por año en un único pago anual, equivalentes a USD 8 mensuales. Los impuestos al consumo que liquida el procesador de pagos se suman al precio y no forman parte del ingreso.

Adopción progresiva. 20 usuarios en el primer año, 60 en el segundo, 120 en el tercero, 200 en el cuarto y 300 en el quinto.

Cómputo temporal. La inversión se efectúa en el momento inicial (Año 0), mientras que los cobros y los costos operativos se imputan al cierre de cada ejercicio anual.

Tabla 5.4: Flujo de fondos proyectado, en dólares (Años 0 a 5)

Año	Usuarios	Ingresos	Costos	Flujo neto	Flujo acumulado
0	—	—	—	−5.340	−5.340
1	20	1.920	−933	987	−4.353
2	60	5.760	−1.145	4.615	262
3	120	11.520	−1.463	10.057	10.319
4	200	19.200	−1.887	17.313	27.632
5	300	28.800	−2.417	26.383	54.015
Total		67.200	−7.845	54.015	

La figura 5.1 representa el mismo flujo: las barras corresponden al flujo neto de cada año y la línea, al acumulado, que cruza el cero a los 1,94 años.

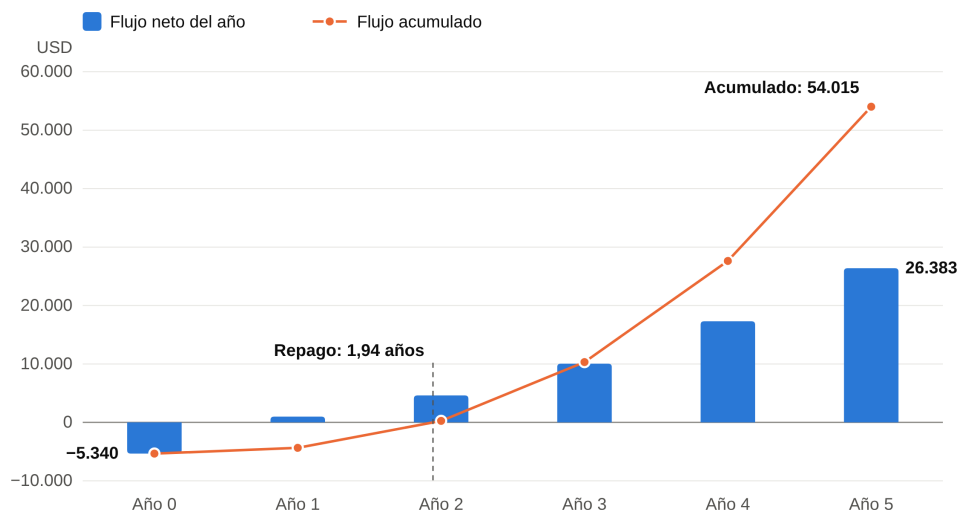


Figura 5.1: Flujo de fondos neto y acumulado, en dólares (Años 0 a 5). El acumulado se vuelve positivo a los 1,94 años

5.5. Tasa de descuento

Se adopta una tasa de descuento del 20 % anual en dólares. La tasa integra los tres componentes conceptuales del costo de capital.

Costo de oportunidad del capital. El rendimiento de un activo libre de riesgo en dólares, representado por los bonos del Tesoro de los Estados Unidos a diez años, se ubica en torno al 3,8 % y el 4,3 % anual. Una inversión diversificada de mercado en el índice S&P 500 promedia históricamente un retorno nominal del 9 % al 10 % anual en moneda dura.

Inflación de la moneda de evaluación. Al confeccionar el flujo en dólares se neutraliza la inflación local en pesos, e incide únicamente la meta de inflación de largo plazo del Comité Federal de Mercado Abierto de la Reserva Federal de los Estados Unidos, fijada en el 2 % anual.

Riesgo específico de etapa temprana. El proyecto consiste en una herramienta de software que parte sin antecedentes comerciales ni base instalada. La literatura de finanzas corporativas aplicada a la valuación de emprendimientos tecnológicos —en particular las bases de datos de costo de capital de Aswath Damodaran, de la Stern School of Business de la Universidad de Nueva York, y los rangos de referencia empleados para inversiones en etapa semilla— establece tasas de corte de entre el 18 % y el 25 % anual en dólares para proyectos tecnológicos incipientes.

Una tasa del 20 % anual reconoce el costo de oportunidad del capital y remunera de manera adecuada la prima de riesgo asociada a un desarrollo sin tracción comercial previa.

5.6. Indicadores financieros

A partir del flujo de fondos de la tabla 5.4, y aplicando la tasa de descuento del 20 %, se obtuvieron los indicadores que expone la tabla 5.5.

Tabla 5.5: Resultados de los indicadores financieros

Indicador	Resultado	Criterio de decisión	Conclusión
Período de repago simple	1,94 años	Menor al horizonte de evaluación	Aceptable
Período de repago descontado	2,23 años	Menor al horizonte de evaluación	Aceptable
Valor Actual Neto (20 %)	USD 23.459	$VAN > 0$	Crea valor
Tasa Interna de Retorno	93,5 %	$TIR > 20 \%$	Supera el mínimo exigido

5.6.1. Lectura de los indicadores

Repago simple (1,94 años). El proyecto recupera la inversión inicial durante el segundo año de operación, únicamente con sus flujos netos de caja.

Repago descontado (2,23 años). Medido en dinero de hoy, el recupero se posterga aproximadamente tres meses y medio respecto del cálculo sin descontar; esa diferencia constituye el precio del tiempo.

Valor Actual Neto (USD 23.459). El proyecto devuelve la inversión, remunera el 20 % anual exigido y deja un excedente cercano a los USD 23.500, medidos en dinero de hoy.

Tasa Interna de Retorno (93,5 %). El rendimiento anual implícito supera ampliamente la tasa exigida; la tasa de corte podría elevarse hasta ese valor antes de que el proyecto dejara de convenir.

5.7. Análisis de sensibilidad y escenarios

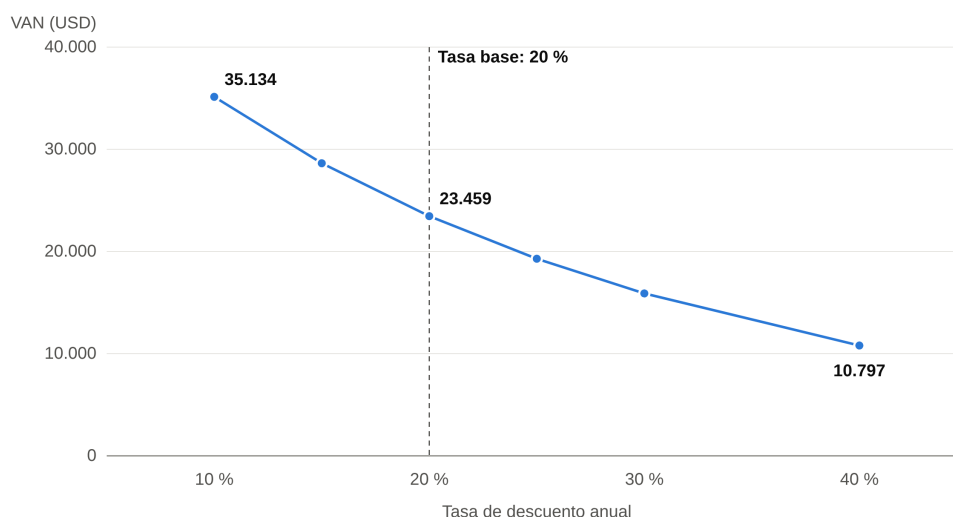
5.7.1. Sensibilidad del Valor Actual Neto a la tasa de descuento

En la tabla 5.6 se evalúa la respuesta del Valor Actual Neto ante incrementos sucesivos de la tasa de corte exigida, entre el 10 % y el 40 % anual.

Tabla 5.6: Sensibilidad del Valor Actual Neto frente a variaciones en la tasa de descuento

Tasa de descuento	Valor Actual Neto (USD)
10 %	35.134
15 %	28.636
20 % (tasa base)	23.459
25 %	19.289
30 %	15.895
40 %	10.797

El Valor Actual Neto se mantiene positivo aun elevando la tasa de corte al 40 %, lo que verifica que la viabilidad del proyecto no depende de una tasa adoptada de forma arbitraria.

**Figura 5.2:** Valor Actual Neto según la tasa de descuento anual

5.7.2. Escenarios de adopción

Siendo la adopción de usuarios la variable de mayor dispersión, la tabla 5.7 modela tres situaciones: pesimista, con la mitad de los usuarios proyectados; base, con la proyección original; y optimista, con un 50 % adicional.

Tabla 5.7: Evaluación de escenarios sobre la base de usuarios

Escenario	Supuesto de usuarios	VAN (USD)	TIR
Pesimista	Mitad de los usuarios proyectados (10 a 150)	7.823	51,7 %
Base	Proyección original adoptada (20 a 300)	23.459	93,5 %
Optimista	50 % adicional sobre la base (30 a 450)	39.096	125,5 %

En el escenario pesimista, captando únicamente la mitad de los usuarios proyectados, el Valor Actual Neto resulta positivo en USD 7.823 y la Tasa Interna de Retorno alcanza el 51,7 %, superando por 31,7 puntos porcentuales la tasa mínima exigida. El proyecto

cuenta, por lo tanto, con margen para absorber desvíos en la captación sin comprometer su conveniencia financiera.

5.8. Conclusión

El análisis muestra que el proyecto resulta viable desde el punto de vista económico: los cuatro indicadores calculados señalan rentabilidad positiva dentro del horizonte de cinco años. No obstante, esa viabilidad queda supeditada a condiciones concretas que corresponde declarar.

5.8.1. Condición crítica

La variable determinante es la captación de usuarios. La inversión de capital es reducida y los costos operativos resultan bajos gracias a que el análisis se ejecuta en el navegador de cada usuario y el servidor solo entrega archivos estáticos. En consecuencia, el riesgo del proyecto no reside en la estructura de costos sino, de manera prácticamente exclusiva, en alcanzar la masa crítica inicial de suscriptores.

5.8.2. Magnitud de la Tasa Interna de Retorno

El valor obtenido para la Tasa Interna de Retorno responde a la naturaleza económica de los productos de software: una vez cubiertas las horas fijas de desarrollo, el costo marginal de atender a un suscriptor adicional resulta prácticamente nulo. La magnitud del indicador debe interpretarse en ese marco y no como una expectativa de rentabilidad garantizada.

5.8.3. Limitaciones del modelo

La evaluación incorpora tres simplificaciones que conviene explicitar. En primer lugar, no se incluye una partida continua de costo de adquisición de clientes más allá del gasto de lanzamiento, por asumirse un crecimiento orgánico a través de comunidades técnicas, foros de seguridad y el registro de paquetes npm. En segundo lugar, no se modeló una tasa explícita de deserción: la cantidad de usuarios de cada período se toma como el saldo neto resultante entre altas y cancelaciones. En tercer lugar, se supone facturación anual: si los suscriptores pagaran mes a mes, cada uno generaría doce transacciones y la comisión anual por usuario pasaría de USD 5,30 a USD 10,80, con lo que el Valor Actual Neto descendería a USD 21.563 y la Tasa Interna de Retorno al 89,1 %; tampoco se incluyeron los cargos adicionales menores que el procesador advierte para algunas transacciones realizadas fuera de Estados Unidos. La incorporación de estos factores reduciría los indicadores obtenidos, si bien el margen que muestra el escenario pesimista sugiere que la conclusión general se mantendría.

5.8.4. Conclusión general

Bajo la condición de que la efectividad técnica del análisis sustente la recomendación entre desarrolladores y se sostenga la adopción mínima correspondiente al escenario pesimista, el proyecto se justifica económicamente para su ejecución.

Este capítulo identifica, evalúa y planifica la respuesta a los riesgos del proyecto en su estado actual: un prototipo funcional publicado, con la validación sobre código real y con usuarios todavía en curso, y con cuatro entregas por delante hasta la entrega final del 17 de noviembre. Reemplaza y amplía la tabla 2.4 del capítulo 2, que registraba tres riesgos técnicos identificados durante la planificación. Varios de los riesgos que se presentan no son hipotéticos: surgieron durante el desarrollo, y en esos casos se indica la evidencia que los originó.

6.1. Enfoque y escalas de evaluación

El marco de referencia es la guía del Project Management Institute (2021), que trata el riesgo como un evento o una condición incierta que, si ocurre, afecta al menos uno de los objetivos del proyecto. Cada riesgo se redacta con una estructura de causa, evento y efecto: «Debido a [causa], podría ocurrir [evento], lo que provocaría [efecto]». La causa es la condición presente que lo origina y sobre la que se puede actuar; el evento, lo que podría suceder; y el efecto, su consecuencia sobre los objetivos.

La probabilidad (P) y el impacto (I) se califican en una escala de 1 a 5, y la exposición (E) se obtiene como su producto, con valores entre 1 y 25. La tabla 6.1 fija los niveles de exposición: baja hasta 5, media entre 6 y 10, y alta desde 12.

Probabilidad. 1 muy improbable; 2 poco probable; 3 posible; 4 probable; 5 muy probable.

Impacto. 1 insignificante, sin efecto visible en los entregables; 2 menor, un retrabajo acotado dentro de la misma semana; 3 moderado, afecta la calidad de un entregable sin comprometer su fecha; 4 importante, compromete una entrega o una hipótesis; 5 severo, compromete la aprobación del proyecto o la defensa.

Tabla 6.1: Matriz de exposición: probabilidad por impacto

Probabilidad / Impacto	1 Insignif.	2 Menor	3 Moderado	4 Importante	5 Severo
5 Muy probable	5	10	15	20	25
4 Probable	4	8	12	16	20
3 Posible	3	6	9	12	15
2 Poco probable	2	4	6	8	10
1 Muy improbable	1	2	3	4	5

Para cada riesgo se elige una de cuatro respuestas: evitar, que elimina la causa, por ejemplo modificando el alcance; mitigar, que reduce la probabilidad o el impacto; transferir, que traslada el riesgo a un tercero con mayor capacidad de absorberlo; y aceptar, que convive con el riesgo en forma documentada, con un responsable y un plan de contingencia. Los riesgos se clasifican además por su naturaleza (alcance, técnica, recursos, terceros, datos y cumplimiento), por la etapa en la que pueden materializarse (antes de arrancar, durante el desarrollo, en el pasaje a producción y durante la operación) y según su origen sea interno o externo al proyecto.

6.2. Supuestos y restricciones

Los supuestos son condiciones que se dan por ciertas sin haberlas verificado; las restricciones, límites que el proyecto no puede modificar. Se registran por separado porque no son riesgos, aunque cada supuesto que deja de cumplirse se convierte en uno. La tabla 6.2 indica con qué riesgo o sección se vincula cada uno.

Tabla 6.2: Supuestos (S) y restricciones (C) del proyecto

ID	Supuesto o restricción	Vínculo
S-01	Los usuarios de la versión web cuentan con un navegador actualizado, con soporte para Web Workers y módulos ES.	R-12
S-02	Las operaciones sensibles que emplean las aplicaciones analizadas coinciden con las registradas en el catálogo: consultas SQL, ejecución de procesos, escritura en el documento y métodos de Mongoose.	R-01, R-08
S-03	Existen aplicaciones Node.js deliberadamente vulnerables, con vulnerabilidades documentadas y una licencia que permite analizarlas, como OWASP NodeGoat (licencia Apache-2.0).	R-02
S-04	Se consiguen entre cinco y ocho desarrolladores dispuestos a participar de la validación de la hipótesis H3 entre el 5 y el 9 de octubre.	R-06
S-05	Vercel mantiene las condiciones y el precio del plan Pro (USD 20 mensuales) durante el horizonte de evaluación.	Capítulo 5 (Tabla 14)
S-06	El procesador de pagos admite vendedores radicados en Argentina y mantiene su comisión del 5 % más USD 0,50 por transacción.	Capítulo 5 (Tabla 14)
C-01	Restricción: el proyecto tiene un único integrante, con una dedicación parcial de tres a cuatro días por semana.	R-11
C-02	Restricción: las fechas son fijas; capítulo 7 el 13 de octubre, capítulo 8 el 20 de octubre, documento integrado el 27 de octubre, sistema congelado el 3 de noviembre y entrega final el 17 de noviembre.	R-03
C-03	Restricción: durante el desarrollo no hay presupuesto; se emplean herramientas libres y planes gratuitos.	R-09
C-04	Restricción: el código analizado no puede salir del equipo del usuario, por lo que el análisis se ejecuta en el navegador o en la interfaz de línea de comandos, sin procesamiento en un servidor.	R-12
C-05	Restricción: el alcance se limita a JavaScript y TypeScript y a las cuatro familias de debilidades del catálogo.	R-01

6.3. Matriz de riesgos

La tabla 6.3 registra los doce riesgos identificados, ordenados por exposición descendente. Cubren las seis naturalezas y las cuatro etapas; tres son de origen externo (R-06, R-07 y R-09) y uno se acepta en forma explícita (R-11). Los ocho primeros, de exposición alta, se desarrollan en la sección 6.4; los restantes se revisan según lo indicado en la sección 6.7.

Tabla 6.3: Matriz de riesgos ordenada por exposición. P: probabilidad; I: impacto; E: exposición

ID	Descripción (causa, evento, efecto)	Naturaleza	Etapas	P	I	E	Respuesta	Resp.	Contingencia
R-01	Debido a que el motor solo resuelve llamadas a funciones declaradas con function e importadas mediante módulos ES, podría ocurrir que en aplicaciones Express escritas con funciones flecha, clases o require el analizador no siga el recorrido del dato, lo que provocaría una exhaustividad baja sobre código real y dejaría sin sustento a la hipótesis H1 fuera del banco sintético.	Técnica	Durante	4	5	20	Mitigar: incorporar al alcance congelado solo las funciones flecha asignadas a constantes y los callbacks de rutas de Express; declarar el resto como limitación.	Autor	Presentar H1 como validada sobre el subconjunto soportado, con la exhaustividad desglosada por construcción.
R-02	Debido a que los advisories de npm corresponden mayormente a bibliotecas publicadas como código compilado o con fallas en su propia lógica de filtrado, podría ocurrir que el corpus de vulnerabilidades reales no tenga casos evaluables por el modelo de origen y destino, lo que provocaría no poder medir el desempeño sobre código real.	Datos	Durante	5	4	20	Mitigar: sumar aplicaciones deliberadamente vulnerables con vulnerabilidades documentadas, como OWASP NodeGoat, etiquetadas antes de analizarlas.	Autor	Declarar la validación sobre código real como preliminar en los capítulos 7 y 8.
R-03	Debido a que cada construcción no soportada que aparece en un caso de prueba se percibe como una mejora pendiente del motor, podría ocurrir que el alcance técnico siga creciendo después del congelamiento, lo que provocaría llegar a la defensa con funciones incompletas y sin tiempo para integrar el documento.	Alcance	Durante	4	4	16	Evitar: cerrar la lista de mejoras el 13 de octubre; lo posterior pasa a trabajo futuro.	Autor	Volver a la última versión etiquetada estable.
R-04	Debido a que los destinos sensibles se reconocen por el nombre de la función invocada, podría ocurrir que llamadas homónimas sin relación con el riesgo se reporten como vulnerabilidades, lo que provocaría falsos positivos que contradicen la propuesta de valor del sistema.	Técnica	Operación	4	3	12	Mitigar: calificar los destinos por el objeto receptor o el módulo importado, por ejemplo exec de child_process.	Autor	Documentar las reglas afectadas como de baja confianza.

ID	Descripción (causa, evento, efecto)	Naturaleza	Etapa	P	I	E	Respuesta	Resp.	Contingencia
R-05	Debido a que las dependencias del proyecto registran vulnerabilidades conocidas, podría ocurrir que el jurado o un usuario las detecte en el repositorio, lo que provocaría una pérdida de credibilidad especialmente grave para una herramienta de seguridad.	Cumplimiento	Pasaje a producción	4	3	12	Mitigar: revisar las alertas, actualizar las dependencias y fijar sus versiones antes del 3 de noviembre.	Autor	Documentar en el README las alertas pendientes y su impacto real.
R-06	Debido a que la validación de H3 depende de desarrolladores externos que participan en forma voluntaria, podría ocurrir que no se reúnan participantes suficientes en la semana prevista, lo que provocaría no poder contrastar la hipótesis sobre la visualización.	Terceros (externo)	Durante	3	4	12	Mitigar: convocatoria anticipada, sesiones remotas de veinte minutos y un protocolo breve.	Autor	Estudio reducido declarado preliminar, complementado con la entrevista a un experto.
R-07	Debido a que existen herramientas consolidadas con análisis de flujo de datos, como Semgrep y CodeQL, gratuitas en determinadas condiciones de uso, podría ocurrir que los desarrolladores no estén dispuestos a pagar una suscripción, lo que provocaría una captación inferior a la proyectada en el capítulo 5.	Recursos (externo)	Operación	3	4	12	Mitigar: posicionar el producto por el análisis desde el navegador sin instalación y por la visualización del recorrido, y relevar la disposición a pagar en la validación con usuarios.	Autor	Reformular el modelo hacia un esquema gratuito con funciones pagas.
R-08	Debido a que el catálogo de orígenes, destinos y sanitizadores depende de APIs de bibliotecas que evolucionan y el autor es su único mantenedor, podría ocurrir que después de la entrega el catálogo quede desactualizado, lo que provocaría falsos negativos crecientes.	Datos	Operación	4	3	12	Transferir: publicar el código con licencia abierta y documentar cómo extender el catálogo en JSON para recibir contribuciones.	Autor	Indicar en el README la fecha de la última revisión del catálogo.
R-09	Debido a que la versión publicada depende de Vercel y de la conectividad del aula, podría ocurrir que el sitio no esté disponible durante la defensa, lo que impediría mostrar el sistema funcionando al jurado.	Terceros (externo)	Pasaje a producción	2	5	10	Mitigar: ejecución local preparada y video de la demostración grabado la semana anterior.	Autor	Mostrar el video y ejecutar la versión local.

ID	Descripción (causa, evento, efecto)	Naturaleza	Etapas	P	I	E	Respuesta	Resp.	Contingencia
R-10	Debido a que el repositorio no declara una licencia, podría ocurrir que el código no pueda reutilizarse ni mantenerse legalmente por terceros, lo que provocaría la falta de continuidad del proyecto e incumpliría un requisito de la cátedra.	Cumplimiento	Antes de arrancar	5	2	10	Evitar: agregar una licencia MIT, compatible con las licencias MIT y Apache-2.0 de las dependencias principales.	Autor	No aplica: la respuesta elimina la causa.
R-11	Debido a que el proyecto tiene un único integrante con dedicación parcial, podría ocurrir una enfermedad o una sobrecarga en las semanas previas a la entrega, lo que provocaría atrasos en las entregas 7 y 8.	Recursos	Durante	2	4	8	Aceptar: no hay reemplazo posible; el margen de cuatro semanas entre la entrega 8 y la final absorbe una semana de atraso, y el avance queda versionado.	Autor	Priorizar el documento sobre nuevas funciones y solicitar una prórroga con evidencia.
R-12	Debido a que el análisis se ejecuta en el navegador con el compilador de TypeScript, un módulo de unos 5,9 MB, y mantiene en memoria el grafo del proyecto, podría ocurrir que en proyectos grandes o en equipos modestos el análisis tarde demasiado o agote la memoria de la pestaña, lo que provocaría que el usuario no obtenga resultados.	Técnica	Operación	2	3	6	Mitigar: excluir dependencias y archivos de más de 1 MB, mostrar el progreso y ofrecer la interfaz de línea de comandos para proyectos grandes.	Autor	Analizar el proyecto con la interfaz de línea de comandos.

6.4. Tratamiento de los riesgos principales

Los ocho riesgos de exposición alta se desarrollan con la evidencia que los sostiene, la respuesta planificada y el indicador con el que se controla su evolución.

6.4.1. R-01: alcance del motor frente a aplicaciones reales

Evidencia. La resolución de llamadas se limita a funciones declaradas por nombre (sección 4.2.5). Sobre el código del propio sistema, el enlace entre archivos resolvió 369 llamadas en 78 archivos, pero los callbacks de rutas de Express y los métodos de clase quedan fuera del recorrido.

Respuesta. Se incorporan al alcance congelado únicamente las funciones flecha asignadas a constantes y los callbacks de rutas de Express, dos formas habituales en las aplicaciones objetivo; el resto de las construcciones se declara como limitación.

Seguimiento. Exhaustividad sobre el corpus de aplicaciones reales, desglosada por construcción sintáctica. Si al 13 de octubre se mantiene por debajo del 50 %, se aplica la contingencia.

6.4.2. R-02: corpus de vulnerabilidades reales

Evidencia. En la validación del capítulo 4 (tabla 4.3), tres de los seis advisories no tenían código fuente analizable en el paquete publicado, y en los restantes la falla residía en la lógica interna de bibliotecas de saneamiento. El riesgo ya se materializó en forma parcial.

Respuesta. Se suma OWASP NodeGoat, una aplicación Node.js construida con fines didácticos para mostrar los riesgos del OWASP Top 10 y distribuida con licencia Apache-2.0. Antes de ejecutar el analizador se etiquetan las vulnerabilidades documentadas que corresponden al catálogo, para que la medición no se ajuste a los resultados.

Seguimiento. Cantidad de casos etiquetados y evaluables al 6 de octubre.

6.4.3. R-03: alcance que no cierra

Evidencia. Durante el desarrollo se sumaron el análisis entre archivos, la versión web y una cuarta familia de debilidades (CWE-943), ninguno de ellos previsto en el alcance definido en el capítulo 2.

Respuesta. El 13 de octubre se cierra la lista de mejoras del motor. Toda mejora posterior se registra como trabajo futuro en el capítulo 8, y desde el 3 de noviembre el repositorio solo admite correcciones de errores.

Seguimiento. Cada entrega se etiqueta en el repositorio, desde entrega-6 hasta entrega-final, lo que permite comparar el alcance entregado con el comprometido.

6.4.4. R-04: falsos positivos por coincidencia de nombres

Evidencia. Al analizar el código del propio sistema, una llamada a `RegExp.prototype.exec` se reportó como inyección de comandos (CWE-78), porque coincide por nombre con el destino `exec` del catálogo.

Respuesta. Se califica cada destino por el objeto receptor o por el módulo del que se importa la función, de modo que `exec` solo se considere sensible cuando proviene de `child_process`.

Seguimiento. Falsos positivos del banco sintético, hoy nulos, y del corpus de aplicaciones reales.

6.4.5. R-05: dependencias con vulnerabilidades conocidas

Evidencia. Al publicar la rama principal, GitHub informó doce alertas de dependencias, tres de ellas críticas, y la instalación en Vercel reportó ocho vulnerabilidades en el árbol de dependencias.

Respuesta. Antes del congelamiento se revisa cada alerta, se actualizan las dependencias afectadas y se distingue entre las que llegan a la versión publicada y las que solo intervienen en el desarrollo.

Seguimiento. Cantidad de alertas críticas y altas abiertas en el repositorio; el objetivo es no tener ninguna al 3 de noviembre.

6.4.6. R-06: participantes para la validación de H3

Evidencia. La hipótesis H3 compara la interpretación del riesgo con y sin visualización, algo que el banco de pruebas no puede medir y que depende de personas externas al proyecto.

Respuesta. La convocatoria se realiza con dos semanas de anticipación, con sesiones remotas de veinte minutos: cada participante localiza el camino vulnerable en un reporte de texto y en el grafo, y se registran el tiempo, el acierto y una valoración en escala de Likert.

Seguimiento. Participantes confirmados al 1 de octubre; con menos de cinco se aplica la contingencia.

6.4.7. R-07: competencia de herramientas gratuitas

Evidencia. Semgrep y CodeQL implementan análisis de flujo de datos (capítulo 3) y se ofrecen sin costo en determinadas condiciones de uso. El capítulo 5 identificó la captación de usuarios como la condición crítica de la viabilidad económica.

Respuesta. El producto se posiciona por el análisis desde el navegador, sin instalación y sin envío del código, y por la visualización interactiva del recorrido del dato. La validación con usuarios incorpora una pregunta sobre la disposición a pagar.

Seguimiento. Resultados de esa pregunta; si la mayoría de los participantes rechaza el precio, se revisa el escenario del capítulo 5.

6.4.8. R-08: mantenimiento del catálogo

Evidencia. El catálogo registra 23 destinos sensibles y 18 sanitizadores en cuatro archivos JSON, y cada nueva versión de una biblioteca puede introducir operaciones que no figuran en él.

Respuesta. Se publica el código con licencia abierta (R-10) y se documenta en el README cómo agregar una entrada al catálogo, para que otros desarrolladores puedan contribuir. La sección 6.6 desarrolla la continuidad del sistema.

Seguimiento. Fecha de la última revisión del catálogo, registrada en el historial del repositorio.

6.5. Riesgos críticos de un proyecto final

La cátedra identifica siete riesgos que con mayor frecuencia hacen fracasar un proyecto final. La tabla 6.4 indica cómo se presenta cada uno en este proyecto y dónde se trata.

Tabla 6.4: Relación entre los riesgos críticos de un proyecto final y los riesgos identificados

Riesgo crítico	Cómo se presenta en este proyecto	Tratamiento
Alcance que nunca cierra	El motor admite mejoras indefinidas: cada construcción del lenguaje no soportada es una candidata.	R-03
Pérdida de la fuente de datos	El corpus de vulnerabilidades reales resultó poco evaluable.	R-02
Equipo que se desarma	El equipo es de una persona y no tiene reemplazo.	R-11
Tecnología que no da	La resolución de llamadas limita el desempeño sobre código real, y el navegador limita el tamaño de proyecto analizable.	R-01, R-12
Inviabilidad de costo	El costo de desarrollo es nulo; en la operación, la viabilidad depende de la captación y no de los costos (capítulo 5).	R-07
Barrera legal o ética	El código analizado no sale del equipo del usuario, lo que evita tratar código de terceros en servidores propios; falta declarar la licencia del repositorio.	R-10
Nadie a cargo	El autor es el único responsable del mantenimiento después de la entrega.	R-08

6.6. Continuidad del sistema después de la entrega

La pregunta sobre quién mantiene el sistema cuando el autor se recibe se responde con tres decisiones. Primero, el código se publicará con licencia MIT en un repositorio público (R-10), lo que permite que cualquier persona lo use, lo modifique y lo continúe. Segundo, la parte que más cambia con el tiempo, las reglas de detección, está separada del motor en los archivos JSON del catálogo, de modo que actualizarla no requiere conocer la implementación del análisis. Tercero, la versión web no tiene componentes de servidor ni base de datos que administrar: el despliegue en Vercel se reconstruye a partir del repositorio, y mantenerla en línea para uso no comercial no tiene costo.

El autor asume el mantenimiento durante el horizonte de evaluación del capítulo 5. Si el proyecto no continúa como producto, queda disponible como herramienta académica de código abierto.

6.7. Seguimiento y actualización

La matriz se revisa en cada entrega, al etiquetar la versión correspondiente en el repositorio: se actualizan la probabilidad y el impacto de cada riesgo, se registran los que se materializaron y se agregan los nuevos. Los riesgos R-09 a R-12 no tienen un tratamiento desarrollado más allá de la respuesta de la tabla 6.3 y se revisan en esas mismas instancias.

Las fechas de control son el 1 de octubre para la convocatoria de participantes (R-06),

el 6 de octubre para el corpus de aplicaciones reales (R-02), el 13 de octubre para el cierre del alcance (R-03) y el 3 de noviembre para las dependencias y la licencia (R-05 y R-10).

Este capítulo presenta la evidencia obtenida durante el desarrollo y la validación de GraphSAST. Se organiza en cinco partes: los requerimientos que el sistema debía cumplir, el proceso con el que se construyó, la evidencia de que funciona, los resultados de las pruebas técnicas y los de la validación con usuarios. Se informan tanto los resultados favorables como los desfavorables, y la lectura de cada hipótesis se reserva para el capítulo 8.

Los objetivos específicos del capítulo 1 se identifican en adelante como OE-1 a OE-6, en el orden en que allí se enuncian: analizador sintáctico (OE-1), construcción del grafo (OE-2), motor de contaminación (OE-3), persistencia en Neo4j (OE-4), interfaz de visualización (OE-5) y validación (OE-6).

7.1. Requerimientos del sistema

Los requerimientos se derivan de los objetivos específicos y de los indicadores de evaluación definidos en el capítulo 2 (tabla 2.1). Cada requerimiento funcional indica el objetivo al que responde y un criterio de aceptación verificable sobre el sistema construido; la verificación de cada criterio se presenta en la sección 7.4.

Tabla 7.1: Requerimientos funcionales

ID	Nombre	Descripción	Prior.	Obj.	Criterio de aceptación
RF-01	Análisis sintáctico	Obtener el árbol de sintaxis abstracta de archivos TypeScript y JavaScript	Alta	OE-1	Los 43 casos del banco sintético se analizan sin errores
RF-02	Construcción del grafo	Generar los cinco tipos de nodo y las cinco relaciones del modelo	Alta	OE-2	Pasan todas las pruebas de representación intermedia y flujo de datos
RF-03	Detección de caminos	Reportar caminos de un origen a un destino sensible sin saneamiento, según el catálogo JSON	Alta	OE-3	Se detectan los 29 casos vulnerables del banco sintético
RF-04	Descarte de saneados	No reportar caminos que atraviesan un saneamiento del catálogo	Alta	OE-3	Ninguno de los 14 casos seguros genera hallazgos
RF-05	Análisis entre archivos	Seguir el dato hacia funciones importadas desde otro archivo del proyecto	Alta	OE-3	En los casos A y B de la validación con usuarios, el hallazgo cruza dos archivos
RF-06	Visualización	Dibujar el grafo, resaltar el camino y vincular cada nodo con su archivo y línea	Alta	OE-5	El camino se resalta y cada nodo muestra su ubicación
RF-07	Carga de proyectos	Analizar en el navegador una carpeta e informar los archivos excluidos	Media	OE-5	El análisis de una carpeta se completa en la versión publicada
RF-08	Reportes	Emitir los hallazgos en texto, JSON y SARIF 2.1.0	Media	OE-6	El SARIF valida contra el esquema oficial
RF-09	Integración continua	Devolver un código de salida distinto de cero ante hallazgos	Media	OE-6	Pasan todas las pruebas de códigos de salida
RF-10	Persistencia en Neo4j	Guardar el grafo y resolver la detección con Cypher	Media	OE-4	Sobre el banco sintético, Cypher y el recorrido en memoria dan los mismos hallazgos
RF-11	API local	Exponer el análisis en una API restringida a 127.0.0.1	Baja	OE-5	Se rechazan las rutas fuera de la carpeta habilitada

Los requerimientos no funcionales se expresan como umbrales sobre los indicadores de la tabla 2.1. El umbral de RNF-02 proviene del seguimiento del riesgo R-01 fijado en el capítulo 6, y el de RNF-07, del riesgo R-05. Los umbrales de RNF-03 a RNF-06 se fijan en este capítulo, dado que el capítulo 2 definió la forma de medición pero no un valor objetivo.

Tabla 7.2: Requerimientos no funcionales

ID	Atributo	Descripción	Prior.	Umbral medible
RNF-01	Corrección sobre casos conocidos	El motor no introduce falsos positivos ni falsos negativos sobre el banco sintético	Alta	FP = 0 y FN = 0 en los 43 casos (precisión y exhaustividad)
RNF-02	Exhaustividad sobre código real	Detección de las vulnerabilidades etiquetadas en una aplicación deliberadamente vulnerable	Alta	Exhaustividad $\geq 50\%$ sobre OWASP NodeGoat (riesgo R-01)
RNF-03	Rendimiento	Tiempo de análisis en la línea de comandos	Media	≤ 1 s por cada 1.000 líneas (indicador de tiempo de análisis)
RNF-04	Confidencialidad	El código analizado no sale del equipo del usuario	Alta	0 solicitudes a otros dominios durante el análisis en la versión web
RNF-05	Portabilidad	La versión web funciona sin instalación	Media	El análisis se completa en un navegador actualizado sin complementos
RNF-06	Mantenibilidad del catálogo	Las reglas se modifican sin cambiar el motor	Media	Agregar un destino o un saneamiento requiere editar solo un archivo JSON
RNF-07	Seguridad de las dependencias	El repositorio no publica dependencias con vulnerabilidades graves	Alta	0 alertas críticas o altas abiertas al 3 de noviembre (riesgo R-05)

7.2. Proceso de desarrollo

El sistema se construyó en forma incremental, siguiendo el orden de etapas previsto en el capítulo 2 (tabla 2.2): primero el análisis sintáctico y la representación intermedia, después el grafo de llamadas y el flujo de datos, a continuación el motor de detección con su persistencia opcional y, por último, la visualización. Cada etapa se cerró con pruebas automatizadas antes de pasar a la siguiente. La tabla 7.3 registra cuándo se completó cada etapa, según los documentos de planificación y validación del repositorio y el historial de cambios.

Tabla 7.3: Etapas de desarrollo completadas

Etapas	Resultado	Registro
Hitos 1 a 3b	Análisis sintáctico, representación intermedia, grafo de llamadas y flujo de datos intra e inter-procedural	12/06/2026
Validación temprana	Verificación de que el grafo contiene los caminos de origen a destino necesarios para la detección	12/06/2026
Hito 5	Motor de detección en memoria, persistencia opcional en Neo4j, catálogo CWE e informes	02/07/2026
Banco sintético	Revisión del corpus de 43 casos y de la metodología de medición	01/09/2026
Cuarta familia	Separación de la inyección NoSQL (CWE-943) de la inyección SQL	15/09/2026
Línea de comandos	Exclusiones, códigos de salida, SARIF, API local y análisis entre archivos	22/09/2026
Versión web	Análisis en el navegador con carga de carpetas y publicación en Vercel	23/09/2026
Código real	Validación sobre vulnerabilidades documentadas del ecosistema npm	23/09/2026

El desarrollo se apartó del plan en dos aspectos. El primero es el alcance: el análisis entre archivos, la versión que se ejecuta en el navegador y la familia CWE-943 no figuraban en el alcance del capítulo 2 y se incorporaron durante el desarrollo, lo que dio origen al riesgo R-03 del capítulo 6. El segundo es el modo de ejecución: el diseño preveía la base de datos de grafos como motor principal, y el sistema final resuelve la detección por defecto con un recorrido en memoria, dejando Neo4j como alternativa (capítulo 4). Desde el hito 5 el motor sobre Neo4j es opcional porque requiere una instancia activa, y en la versión web, que se ejecuta en el navegador, el recorrido en memoria es el único disponible.

El diseño del sistema, su organización en paquetes y el modelo de datos del grafo se describen en el capítulo 4. La integración entre el motor y la interfaz se resolvió con un punto de entrada del motor sin dependencias de Node.js, que la versión web ejecuta en un proceso en segundo plano del navegador; de este modo, la línea de comandos y la interfaz web usan el mismo código de análisis y las pruebas del motor cubren a ambas.

7.3. Evidencia de implementación

El sistema se encuentra publicado en <https://graph-sast.vercel.app/> y su código fuente en el repositorio indicado en el anexo I. La versión que corresponde a esta entrega se identifica con la etiqueta [entrega-7] del repositorio. Las capturas del sistema en funcionamiento se presentan en el capítulo 4: la detección de un recorrido vulnerable, el descarte del mismo recorrido saneado, el panel de métricas y el análisis de un proyecto cargado como carpeta, en el que el hallazgo cruza tres archivos.

[Agregar una captura del caso A de la validación con usuarios cargado en la versión publicada, con el camino entre los dos archivos resaltado.]

7.4. Pruebas técnicas

7.4.1. Pruebas automatizadas

La suite automatizada comprende 247 pruebas en 27 archivos, ejecutadas con Vitest: 219 corresponden al motor y 28 a la interfaz web. La tabla 7.4 las agrupa por etapa del procesamiento. En la ejecución del 24 de septiembre de 2026 todas las pruebas resultaron satisfactorias. La paridad entre el motor Cypher y el recorrido en memoria (RF-10) no forma parte de la suite, porque requiere una instancia activa de Neo4j. [Resultado de la verificación de paridad con Neo4j sobre el banco sintético.]

Tabla 7.4: Pruebas automatizadas por etapa

Etapa	Pruebas	Requerimiento
Análisis sintáctico	2	RF-01
Representación intermedia y grafo de llamadas	14	RF-02
Flujo de datos intra e inter-procedural y ámbitos	34	RF-02
Motor de contaminación y destinos XSS	26	RF-03, RF-04
Catálogo de reglas	8	RNF-06
Veredicto y selección de motor	18	RF-03
Análisis entre archivos	22	RF-05
Escaneo de archivos y exclusiones	31	RF-07, RF-08
Informes de texto, JSON y SARIF	21	RF-08
Códigos de salida	9	RF-09
API local	19	RF-11
Generación de consultas Cypher	2	RF-10
Evaluación y banco de pruebas	7	RNF-01
Punto de entrada del motor	6	RF-01
Interfaz web	28	RF-06, RF-07
Total	247	

7.4.2. Banco de pruebas sintético

Diseño. El banco se compone de 43 casos de verdad conocida, 29 vulnerables y 14 seguros, con un total de 198 líneas de código. Cada caso declara si es vulnerable, la familia de debilidad y la cantidad esperada de hallazgos. Los casos cubren recorridos dentro de una función y entre funciones, entradas propias de Express (parámetros, cuerpo, encabezados y cookies), asignaciones, desestructuración, literales de plantilla, funciones flecha, métodos de clase y componentes JSX. Entre los casos seguros se incluyen tres contruidos contra la detección por nombres: funciones llamadas `evaluatePrice`, `myExecutor` y `spawnConfetti`, cuyos nombres contienen los de destinos sensibles del catálogo (`eval`, `exec` y `spawn`) sin serlo. En sentido inverso, dos casos vulnerables verifican que el saneamiento se reconoce por su posición en el camino y no por el nombre: uno aplica el saneamiento en una rama paralela que no protege al destino, y otro usa una función cuyo nombre contiene `escape` sin sanear el dato.

Tabla 7.5: Composición del banco sintético por familia

Familia	Vulnerables	Seguros
CWE-89, inyección SQL	21	5
CWE-79, Cross-Site Scripting	3	3
CWE-78, inyección de comandos	1	0
CWE-943, inyección NoSQL	4	0
Sin familia (nombres engañosos y otros negativos)	0	6
Total	29	14

Resultados. La tabla 7.6 presenta la matriz de confusión. El sistema clasificó correctamente los 43 casos: detectó los 29 vulnerables y no emitió hallazgos sobre los 14 seguros, con una precisión, una exhaustividad y un F_1 de 1,00. El análisis completo del banco demandó 52 ms (mediana de cinco ejecuciones), es decir, 0,27 ms por línea, lo que equivale a 0,27 s por cada 1.000 líneas y cumple el umbral de RNF-03.

Tabla 7.6: Matriz de confusión sobre el banco sintético (43 casos)

	Reportado vulnerable	No reportado
Caso vulnerable	29 (TP)	0 (FN)
Caso seguro	0 (FP)	14 (TN)

Alcance del resultado. El banco fue construido por el autor y se limita a construcciones que el analizador cubre, de modo que un resultado perfecto sobre él no constituye evidencia de desempeño sobre código real. Su función es de regresión: verificar que cada cambio del motor mantiene la detección de los casos conocidos (RNF-01). La distribución es además desigual: 26 de los 43 casos corresponden a inyección SQL y solo uno a inyección de comandos.

7.4.3. Comparación con una línea de base

Las hipótesis H1 y H2 comparan el análisis sobre el grafo con la detección por coincidencia de patrones. Para medir esa diferencia se aplica sobre los mismos casos una línea de base que [marca como vulnerable toda llamada cuyo nombre coincide con un destino del catálogo y que recibe al menos un argumento no literal, sin seguir el flujo del dato ni reconocer saneamientos].

Tabla 7.7: Resultados de la línea de base y de GraphSAST sobre los mismos casos

Corpus	Detector	FP	FN	Precisión	Exhaustividad	F_1
Banco sintético	Línea de base	[..]	[..]	[..]	[..]	[..]
Banco sintético	GraphSAST	0	0	1,00	1,00	1,00
NodeGoat	Línea de base	[..]	[..]	[..]	[..]	[..]
NodeGoat	GraphSAST	[..]	[..]	[..]	[..]	[..]

[Lectura de la comparación: en qué casos difieren los dos detectores y por qué.]

7.4.4. Vulnerabilidades documentadas en paquetes de npm

Para contrastar el sistema con vulnerabilidades reales se analizaron seis advisories del ecosistema npm publicados en la base de datos de GitHub, relevados el 1 de septiembre de 2026 y congelados para que la medición sea reproducible. Para cada caso se analizaron la versión vulnerable y la corregida del paquete, contando solo los hallazgos de la familia del advisory. El detalle se presenta en el capítulo 4.

El sistema detectó uno de los seis casos (16,7 %) y ninguno con discriminación: el único hallazgo, en DOMPurify, se mantiene en la versión corregida, por lo que no corresponde a la vulnerabilidad reportada. En dos paquetes el registro de npm solo contiene código compilado; en otro, el paquete publicado es un instalador de tres archivos; y en los restantes la falla reside en la lógica interna de filtrado de bibliotecas de saneamiento, que no es un recorrido de un origen a un destino. El corpus, por lo tanto, no permitió evaluar el modelo de amenaza del analizador, y este resultado dio lugar a la respuesta al riesgo R-02: sumar una aplicación deliberadamente vulnerable.

7.4.5. Aplicación deliberadamente vulnerable: OWASP NodeGoat

Diseño. OWASP NodeGoat es una aplicación Node.js construida con fines didácticos para mostrar los riesgos del OWASP Top 10, distribuida con licencia Apache-2.0. Antes de ejecutar el analizador se etiquetan las vulnerabilidades documentadas que corresponden a las familias del catálogo, de modo que la medición no se ajuste a los resultados. [Versión o commit analizado y fecha.]

Resultados. [Cantidad de vulnerabilidades etiquetadas por familia, hallazgos, matriz de confusión y exhaustividad desglosada por construcción sintáctica (riesgo R-01).]

7.4.6. Rendimiento

Sobre el banco sintético el análisis demandó 0,27 ms por línea en la línea de comandos. En la versión web, el análisis de un proyecto de 78 archivos y unas 8.500 líneas demandó alrededor de 2,8 s en el equipo de desarrollo. [Tiempo de análisis de NodeGoat en la línea de comandos y en la versión web.]

7.5. Validación con usuarios y con un experto

Diseño. La hipótesis H3 se evalúa con un estudio intra-sujeto: cada participante resuelve un caso con el reporte de texto de la línea de comandos y otro con la visualización del grafo, y el orden de las condiciones y de los casos se alterna entre participantes. Los dos casos tienen la misma estructura, dos archivos con un camino vulnerable que los cruza y un endpoint saneado, y difieren en la debilidad: inyección SQL en el caso A e inyección de comandos en el caso B. Por cada tarea se registran el acierto en cuatro preguntas, el tiempo y dos valoraciones en escala de 1 a 5. El protocolo completo y el cuestionario se presentan en el apéndice A. De forma complementaria se entrevistó a un desarrollador con experiencia en seguridad de aplicaciones (apéndice B).

Participantes. [Cantidad, experiencia con JavaScript, período de las sesiones.]

Tabla 7.8: Resultados por condición en la validación con usuarios

Indicador	Reporte de texto	Grafo
Acierto, mediana (0 a 4)	[..]	[..]
Tiempo, mediana (s)	[..]	[..]
L1 comprensión del recorrido, mediana (1 a 5)	[..]	[..]
L2 claridad de la corrección, mediana (1 a 5)	[..]	[..]
Participantes con mejor resultado en la condición	[..]	[..]

[Preferencia declarada, respuestas sobre la disposición a pagar USD 8 mensuales (riesgo R-07) y observaciones de las sesiones.]

Entrevista. [Síntesis de la entrevista por tema.]

Tabla 7.9: Traducción de la retroalimentación en decisiones de diseño

Fuente	Hallazgo	Decisión	Estado
Sesiones (n=[..])	[...]	[...]	[...]
Entrevista	[...]	[...]	[...]

7.6. Resultados desfavorables y su análisis

Validación sobre paquetes de npm. Como se expuso en la sección 7.4, solo uno de seis advisories produjo un hallazgo, y ese hallazgo no discrimina entre la versión vulnerable y la corregida. El resultado no confirma ni refuta la efectividad del sistema, porque las fallas del corpus no son recorridos de un origen a un destino, pero deja sin evidencia sobre código real a la validación prevista en el capítulo 2 hasta contar con los resultados de NodeGoat.

Falsos positivos por coincidencia de nombres. Al analizar el código del propio sistema, una llamada a `RegExp.prototype.exec` se reportó como inyección de comandos, porque coincide por nombre con el destino `exec` del catálogo. El banco sintético no detectó el problema porque sus casos engañosos usan nombres que contienen al de un destino, como `myExecutor`, y no el mismo nombre invocado sobre otro objeto. [Estado de la corrección: calificar el destino por el módulo del que se importa (riesgo R-04).]

Cobertura del análisis entre archivos. El dato se sigue entre archivos solo cuando la función invocada se declara con `function` y se importa con módulos ES y ruta relativa. Dentro de un mismo archivo, en cambio, el banco sintético incluye casos con funciones flecha y con un método de clase que se detectan correctamente. La limitación afecta, por lo tanto, a las aplicaciones Express organizadas en varios archivos con funciones flecha o con `require`, que es la forma habitual de NodeGoat. [Efecto medido sobre NodeGoat.]

7.7. Síntesis de la evidencia

La tabla 7.10 resume qué evidencia se obtuvo para cada hipótesis. La respuesta a cada una se desarrolla en el capítulo 8.

Tabla 7.10: Evidencia obtenida por hipótesis

Hipótesis	Evidencia obtenida	Sección
H1	Banco sintético: 29 de 29 casos vulnerables detectados, incluidos recorridos entre funciones y entre archivos. [Comparación con la línea de base y resultados de NodeGoat.]	7.4
H2	Banco sintético: 0 falsos positivos en 14 casos seguros, tres de ellos saneados y tres con nombres engañosos; dos casos vulnerables con saneamientos que no protegen el destino se detectan. Un falso positivo por coincidencia de nombres fuera del banco. [Falsos positivos de la línea de base.]	7.4, 7.6
H3	[Resultados de las sesiones y de la entrevista.]	7.5

Implicancias, conclusiones y recomendaciones

[Párrafo de apertura: la pregunta de investigación del capítulo 1 y cómo se organiza la respuesta.]

8.1. Respuesta a las hipótesis

H1. La representación del código como grafo de flujo de datos permite identificar vulnerabilidades con mayor precisión que la coincidencia de patrones. [Respuesta (afirmativa, negativa o con matices) con la cifra de la tabla 7.7 que la sostiene y el contraejemplo que la limita.]

H2. El análisis de contaminación sobre el grafo reduce los falsos positivos respecto de la detección por expresiones regulares. [Respuesta con los falsos positivos de la línea de base y de GraphSAST, y el falso positivo por coincidencia de nombres de la sección 7.6.]

H3. La visualización del recorrido facilita la interpretación del riesgo frente al listado en texto. [Respuesta con los resultados de la tabla 7.8, declarada preliminar por el tamaño de la muestra.]

8.2. Cumplimiento de los objetivos

Tabla 8.1: Grado de cumplimiento de los objetivos específicos

Objetivo	Evidencia	Grado	Observación
OE-1	RF-01; sección 7.4	[..]	El análisis sintáctico se apoya en ts-morph, sin un analizador propio (capítulo 4)
OE-2	RF-02; tabla 7.4	[..]	Cinco categorías de nodo y cinco de arista, como se definió
OE-3	RF-03 a RF-05; tabla 7.6	[..]	Cuatro familias en lugar de tres: se sumó CWE-943
OE-4	RF-10	[..]	Neo4j opera como motor opcional; el recorrido en memoria es el modo por defecto
OE-5	RF-06, RF-07; capítulo 4	[..]	La visualización se publicó como aplicación web que analiza en el navegador
OE-6	Secciones 7.4 y 7.5	[..]	[Estado de la validación sobre código real y con usuarios]

8.3. Implicancias

Técnicas. [Qué muestran los resultados sobre la formulación de la seguridad como accesibilidad en un grafo: dónde funcionó y dónde la resolución de llamadas limita el enfoque.]

Sociales. [Análisis en el navegador sin enviar el código a un servidor; acceso a una herramienta de este tipo para desarrolladores independientes y equipos chicos; vínculo con el aseguramiento de la calidad.]

Económicas. [Relación con el capítulo 5: la captación de usuarios como condición crítica y lo que respondieron los participantes sobre la disposición a pagar.]

8.4. Limitaciones

Técnicas. El análisis entre archivos solo sigue funciones declaradas con `function` e importadas con módulos ES y ruta relativa. Los destinos sensibles se reconocen por el nombre de la función invocada. [Estado final tras el congelamiento del 3 de noviembre.]

De datos. El banco sintético fue construido por el autor y concentra 26 de sus 43 casos en inyección SQL. El corpus de advisories de npm resultó no evaluable por el modelo de amenaza del analizador. [Tamaño y representatividad del corpus de NodeGoat.]

De validación. [Tamaño y origen de la muestra de participantes; autor en el doble rol de desarrollador y facilitador; casos de dos archivos.]

De mercado. [Competencia de herramientas gratuitas en determinadas condiciones de uso (riesgo R-07) y supuestos de captación del capítulo 5.]

8.5. Recomendaciones

[Recomendaciones accionables para la práctica profesional (por ejemplo, en qué etapa del desarrollo incorporar el análisis) y para el desarrollo tecnológico de la herramienta.]

8.6. Trabajo futuro

[Mejoras que quedaron fuera del cierre de alcance del 13 de octubre (riesgo R-03): resolución de funciones flecha, métodos de clase y `require` entre archivos; calificación de destinos por módulo; ampliación del catálogo; validación con una muestra mayor y con proyectos reales de mayor tamaño.]

8.7. Contribución al conocimiento

[Qué se sabe después de este trabajo que no se sabía antes. El aporte no es la técnica, que existe en CodeQL, Semgrep y Joern (capítulo 3), sino la implementación propia de la cadena completa y su medición, incluidos los resultados desfavorables.]

Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Addison-Wesley.
- Balbi, P. (2023, septiembre). Defensa de tesis de licenciatura en Ciencias de la Computación. <https://www.dc.uba.ar/event/defensa-tesis-licenciatura-pablo-balbi/>
- Banco Central de la República Argentina. (s.f.). *Tipo de cambio de referencia, Comunicación «A» 3500*. <https://www.bcra.gob.ar/>
- Brito, T., Ferreira, M., Monteiro, M., Lopes, P., Barros, M., Santos, J., & Santos, N. (2023). *Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages*. arXiv: 2301.05097.
- CESSI. (s.f.). *Observatorio Permanente de la Industria del Software y Servicios Informáticos (OPSSI)*. <https://www.cessi.org.ar/opssi>
- Damodaran, A. (s.f.-a). *Damodaran Online: datos de costo de capital*. Stern School of Business, New York University. <https://pages.stern.nyu.edu/~adamodar/>
- Damodaran, A. (s.f.-b). *Historical Returns for the US*. https://pages.stern.nyu.edu/~adamodar/New_Home_Page/datafile/histretSP.html
- Denning, D. E. (1976). A lattice model of secure information flow. *Communications of the ACM*, 19(5), 236-243.
- Diehl, S. (2007). *Software Visualization: Visualizing the Structure, Behaviour, and Evolution of Software*. Springer.
- Dutta, S., Garbervetsky, D., Lahiri, S. K., & Schäfer, M. (2022). InspectJS: Leveraging Code Similarity and User-Feedback for Effective Taint Specification Inference for JavaScript. *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '22)*, 165-174. <https://doi.org/10.1145/3510457.3513048>
- Federal Reserve. (s.f.). *Why does the Federal Reserve aim for inflation of 2 percent over the longer run?* https://www.federalreserve.gov/faqs/economy_14400.htm
- GitHub. (s.f.-a). *CodeQL: About data flow analysis*. Consultado 10 de junio de 2026, desde <https://codeql.github.com/docs/writing-codeql-queries/about-data-flow-analysis/>

- GitHub. (s.f.-b). *GitHub Advisory Database*. <https://github.com/advisories>
- GitHub. (s.f.-c). *Octoverse*. <https://octoverse.github.com/>
- Joern. (s.f.). *Joern Documentation*. Consultado 10 de junio de 2026, desde <https://docs.joern.io/>
- Lemon Squeezy. (s.f.-a). *License API*. <https://docs.lemonsqueezy.com/help/licensing/license-api>
- Lemon Squeezy. (s.f.-b). *Pricing*. <https://www.lemonsqueezy.com/pricing>
- Li, S., Kang, M., Hou, J., & Cao, Y. (2022). Mining Node.js Vulnerabilities via Object Dependence Graph and Query. *31st USENIX Security Symposium*, 143-160.
- McGraw, G. (2006). *Software Security: Building Security In*. Addison-Wesley Professional.
- MITRE. (s.f.). *CWE Top 25 Most Dangerous Software Weaknesses*. Consultado 10 de junio de 2026, desde <https://cwe.mitre.org/top25/>
- Nielson, F., Nielson, H. R., & Hankin, C. (1999). *Principles of Program Analysis*. Springer. <https://doi.org/10.1007/978-3-662-03811-6>
- NIST. (s.f.-a). *Secure Software Development Framework (SSDF), SP 800-218*. Consultado 10 de junio de 2026, desde <https://csrc.nist.gov/pubs/sp/800/218/final>
- NIST. (s.f.-b). *Software Assurance Reference Dataset (SARD)*. Consultado 10 de junio de 2026, desde <https://samate.nist.gov/SARD/>
- npm. (s.f.). *Registro público de paquetes*. <https://registry.npmjs.org/>
- OpenQube. (s.f.). *Resultados de la encuesta de sueldos de sysarmy*. <https://sueldos.openqube.io/>
- OWASP Foundation. (s.f.-a). *OWASP Benchmark Project*. Consultado 10 de junio de 2026, desde <https://owasp.org/www-project-benchmark/>
- OWASP Foundation. (s.f.-b). *OWASP NodeGoat*. <https://github.com/OWASP/NodeGoat>
- OWASP Foundation. (s.f.-c). *Source Code Analysis Tools*. Consultado 10 de junio de 2026, desde https://owasp.org/www-community/Source_Code_Analysis_Tools
- Paddle. (s.f.). *Pricing*. <https://www.paddle.com/pricing>
- Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide) (7.ª ed.)*.
- Robinson, I., Webber, J., & Eifrem, E. (2013). *Graph Databases*. O'Reilly Media.
- Sabelfeld, A., & Myers, A. C. (2003). Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1), 5-19.
- SADIO. (s.f.). *Simposio Argentino de Ingeniería de Software (ASSE), Jornadas Argentinas de Informática (JAIIO)*. Consultado 10 de junio de 2026, desde <https://jaiio53.clei.org/simposios/asse/>

- Semgrep. (s.f.). *Taint mode documentation*. Consultado 10 de junio de 2026, desde <https://semgrep.dev/docs/writing-rules/data-flow/taint-mode/>
- SlashData. (s.f.). *Developer Nation*. <https://www.slashdata.co/research/developer-nation>
- ts-morph. (s.f.). *Documentation*. Consultado 10 de junio de 2026, desde <https://ts-morph.com/>
- U.S. Department of the Treasury. (s.f.). *Daily Treasury Rates*. https://home.treasury.gov/resource-center/data-chart-center/interest-rates/TextView?type=daily_treasury_yield_curve
- Vercel. (s.f.-a). *Fair Use Guidelines*. <https://vercel.com/docs/limits/fair-use-guidelines>
- Vercel. (s.f.-b). *Pricing*. <https://vercel.com/pricing>
- Villan, M. A., & Melani, M. I. (2022). Análisis de vulnerabilidades en videojuegos basados en NFT y criptomonedas. *XXIV Workshop de Investigadores en Ciencias de la Computación (WICC 2022, Mendoza)*, 589-591. <http://sedici.unlp.edu.ar/handle/10915/144933>
- Yamaguchi, F., Golde, N., Arp, D., & Rieck, K. (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. *IEEE Symposium on Security and Privacy*, 590-604.

Apéndices

A.1. Introducción

La encuesta forma parte de la validación de la hipótesis H3: la visualización del recorrido de los datos facilita al desarrollador la comprensión del riesgo frente a un listado de errores en texto. Se aplica dentro de una sesión en la que cada participante resuelve dos tareas, una con el reporte de texto de la línea de comandos y otra con la visualización del grafo, y responde las mismas preguntas en ambas. Incluye además una pregunta sobre la disposición a pagar el precio definido en el capítulo 5, que da seguimiento al riesgo R-07.

A.2. Metodología

Diseño. Estudio intra-sujeto con dos casos equivalentes y orden contrabalanceado. El caso A contiene una inyección SQL y el caso B una inyección de comandos; ambos tienen dos archivos, un camino vulnerable que los cruza pasando por tres funciones y un endpoint saneado que funciona como distractor. Cada participante recibe uno de cuatro órdenes (tabla A.1), asignados en forma rotativa.

Tabla A.1: Órdenes de presentación de las condiciones y los casos

Orden	Primera tarea	Segunda tarea
O1	Texto, caso A	Grafo, caso B
O2	Grafo, caso A	Texto, caso B
O3	Texto, caso B	Grafo, caso A
O4	Grafo, caso B	Texto, caso A

Población y canal. Personas que programan en JavaScript o TypeScript con regularidad y que no participaron del desarrollo del sistema. La convocatoria se realizó por mensaje directo, con sesiones remotas de veinte minutos. [Período de aplicación, cantidad de personas convocadas y cantidad de sesiones realizadas.]

Registro. Por cada tarea se registran el acierto en las preguntas P1 a P4 (de 0 a 4, según una clave de corrección fijada antes de las sesiones), el tiempo hasta responder P4, con un límite de seis minutos, y las valoraciones L1 y L2.

A.3. Presentación de la encuesta

Texto leído al inicio de cada sesión:

La sesión dura veinte minutos. Vas a mirar dos fragmentos de código con los resultados de una herramienta y responder cuatro preguntas sobre cada uno. Se evalúa la herramienta, no a vos. Registro el tiempo y tus respuestas sin tu nombre, y los resultados se usan solo en mi proyecto final. Podés dejar la sesión cuando quieras. ¿Estás de acuerdo?

A.4. Cuestionario

Preguntas de cada tarea (abiertas; entre corchetes, el nombre que corresponde a cada caso):

- P1. ¿Qué dato ingresado por el usuario llega a la operación peligrosa?
- P2. ¿Por qué funciones pasa ese dato, en orden, hasta llegar a ella?
- P3. ¿El endpoint [verCliente / diagnostico] tiene la misma vulnerabilidad? ¿Por qué?
- P4. Si tuvieras que corregir la vulnerabilidad con un solo cambio, ¿en qué archivo y línea lo harías?

Valoración de cada tarea (escala de 1, totalmente en desacuerdo, a 5, totalmente de acuerdo):

- L1. Entendí por dónde viaja el dato desde la entrada hasta la operación peligrosa.
- L2. Tengo claro dónde aplicar la corrección.

Preguntas finales:

- F1. ¿Con cuál de las dos formas preferirías trabajar: texto, grafo o indistinto? ¿Por qué? (opción única y abierta)
- F2. Una herramienta así, que corre en el navegador sin enviar tu código a un servidor, ¿la pagarías a USD 8 por mes? Sí / No / Solo con otras funciones, ¿cuáles? (opción única y abierta)
- F3. ¿Algo que te haya costado o que agregarías? (abierta)

A.5. Casos utilizados

```
1 import { buscarPedidos, buscarCliente } from "../pedidos.js";
2
3 export function listarPedidos(req, res) {
4   const estado = req.query.estado;
5   const filtro = armarFiltro(estado);
6   res.json(buscarPedidos(filtro));
7 }
8
9 export function verCliente(req, res) {
```

```

10   const id = sanitize(req.params.id);
11   res.json(buscarCliente(id));
12 }
13
14 function armarFiltro(valor) {
15   return "estado = '" + valor + "'";
16 }

```

Código A.1: Caso A, archivo rutas.js

```

1 export function buscarPedidos(filtro) {
2   return db.query("SELECT * FROM pedidos WHERE " + filtro);
3 }
4
5 export function buscarCliente(id) {
6   return db.query("SELECT * FROM clientes WHERE id = " + id);
7 }

```

Código A.2: Caso A, archivo pedidos.js

El caso B tiene la misma estructura: `exportarReporte` recibe `req.body.carpeta`, la pasa por `armarRuta` y llega a `execSync` en `comprimirCarpeta`; el distractor diagnóstico pasa el host por `validate`. Los archivos de ambos casos y los reportes de texto entregados a los participantes se encuentran en la carpeta `docs/validacion/h3/` del repositorio (anexo I).

B

Entrevista a [nombre y título de la persona entrevistada]

B.1. Contexto de la entrevista

[Fecha, modalidad, duración y objetivo.]

B.2. Datos de la persona entrevistada

[Formación, cargo actual y experiencia relevante.]

B.3. Transcripción de la entrevista

B.3.1. [Tema 1]

Entrevistador [Pregunta.]

Entrevistado [Respuesta.]

B.3.2. [Tema 2]

[...]

C.1. Versiones

La tabla C.1 registra las versiones instaladas según el archivo de bloqueo de dependencias del repositorio al 24 de septiembre de 2026, con las que se obtuvieron los resultados del capítulo 7.

Tabla C.1: Versiones de las dependencias principales

Componente	Versión	Uso
GraphSAST	0.1.0	Versión del motor, del CLI y del SARIF
Node.js	26.7.0	Entorno de ejecución de la línea de comandos
TypeScript	5.9.3	Lenguaje y compilador
ts-morph	23.0.0	Análisis sintáctico
Cytoscape.js	3.34.0	Visualización del grafo
Vite	5.4.21	Empaquetado de la versión web
Vitest	2.1.9	Pruebas automatizadas
neo4j-driver	6.2.0	Cliente de Neo4j
Neo4j	5 Community	Base de datos de grafos (imagen neo4j:5-community)

C.2. Parámetros del análisis

Tabla C.2: Parámetros del motor con sus valores por defecto

Parámetro	Valor	Efecto
--max-depth	15	Longitud máxima del camino entre origen y destino
--max-file-bytes	1.000.000	Tamaño máximo de un archivo analizado
Análisis entre archivos	Activo	Se desactiva con --no-cross-file

C.3. Catálogo de detección

El catálogo se compone de cuatro archivos JSON, uno por familia, en la carpeta catalog/ del repositorio. Cada archivo declara los orígenes, los destinos sensibles y los saneamientos de la familia; su contenido se resume en la tabla de debilidades del capítulo 4.

C.4. Reproducción de los resultados

```
1 npm install
2 npm test           # suite automatizada (247 pruebas)
3 npm run benchmark  # banco sintetico (43 casos)
4 npm run cve:run     # advisories de npm
5 npm run scan -- scan docs/validacion/h3/caso-a --verbose
```

Código C.1: Comandos para reproducir las mediciones del capítulo 7

[Comando y versión de NodeGoat, y comando de la línea de base.]

C.5. Variables de entorno

El motor sobre Neo4j se habilita desde la API local (graphsast serve) cuando está definida `NEO4J_URI` y apunta a la propia máquina; si apunta a otro equipo, se ignora y se usa el recorrido en memoria, para que el código analizado no salga del equipo. La instancia local se levanta con `npm run neo4j:up`.

Tabla C.3: Variables de entorno

Variable	Propósito	Valor por defecto
NEO4J_URI	Dirección de la instancia de Neo4j; su presencia habilita el motor Cypher	Sin definir
NEO4J_USER	Usuario de la instancia	neo4j
NEO4J_PASSWORD	Contraseña de la instancia	La de desarrollo de <code>docker-compose.yml</code>
CVE_JSON	Archivo donde la validación con advisories escribe el informe en JSON	Sin definir

Anexos

I

Repositorios de código fuente

I.1. GraphSAST



MaxiZk/graphsast

Monorepo en TypeScript con dos paquetes: *core*, que contiene el motor de análisis, la línea de comandos y la API local, y *viz*, que contiene la interfaz web. Incluye el catálogo de detección en *catalog/*, el banco sintético, el evaluador de advisories y los materiales de la validación con usuarios en *docs/validacion/h3/*. La instalación y la ejecución se describen en el archivo *README.md*. La versión publicada se encuentra en <https://graph-sast.vercel.app/>.

[Licencia (riesgo R-10) y etiqueta de la versión entregada.] Estado: activo.

I.2. OWASP NodeGoat



OWASP/NodeGoat

Aplicación Node.js deliberadamente vulnerable, construida por la fundación OWASP con fines didácticos y distribuida con licencia Apache-2.0. Se utiliza como corpus de validación sobre código real en el capítulo 7. [Commit analizado.]

Glosario

accesibilidad (reachability)	Propiedad de un grafo dirigido que indica si existe una secuencia de aristas que conecte un nodo de origen con un nodo de destino.
análisis de contaminación (taint analysis)	Método que verifica si datos provenientes de fuentes no confiables pueden alcanzar operaciones sensibles sin una sanitización intermedia.
deuda técnica de seguridad	Costo diferido y riesgo acumulado en un sistema por decisiones de diseño o deficiencias de seguridad no resueltas en el código fuente.
exhaustividad (recall)	Proporción de vulnerabilidades reales detectadas sobre el total de vulnerabilidades existentes.
exposición al riesgo	Producto de la probabilidad de ocurrencia de un riesgo por su impacto. Se usa para ordenar la matriz de riesgos y decidir cuáles se tratan primero.
falso negativo	Vulnerabilidad real presente en el código fuente que el analizador no detecta.
falso positivo	Alerta emitida por el analizador sobre un fragmento de código seguro o correctamente sanitizado.
grafo de flujo de datos	Grafo dirigido cuyos vértices representan variables, operaciones e identificadores, y cuyas aristas modelan la propagación de los valores.
precisión (precision)	Proporción de vulnerabilidades reales sobre el total de alertas emitidas por el analizador.
producto mínimo viable	Versión funcional de un producto con el conjunto mínimo de características que entrega valor real a los usuarios y permite aprender de su uso. A diferencia de un prototipo, no simula: funciona.
prototipo	Versión preliminar de un sistema construida para validar una idea, un flujo o una interfaz. Puede simular partes del comportamiento y no está pensada para operar en producción.

sanitizer	Operación que valida, filtra o codifica un dato y le quita la condición de contaminado.
sink	Operación sensible en la que el uso de un dato no confiable produce una vulnerabilidad, como una consulta a una base de datos o la ejecución de un comando.
source	Punto por el que ingresa al programa información externa no controlada, como los parámetros de una petición HTTP.
vulnerabilidad de arquitectura	Defecto de seguridad que no se manifiesta en una sentencia aislada sino en la forma en que los componentes se vinculan y en el recorrido de la información entre ellos.

Siglas

API	Application Programming Interface.
AST	Abstract Syntax Tree, árbol de sintaxis abstracta.
CFG	Control Flow Graph, grafo de flujo de control.
CI/CD	Continuous Integration / Continuous Deployment.
CLI	Command Line Interface, interfaz de línea de comandos.
CWE	Common Weakness Enumeration.
DFG	Data Flow Graph, grafo de flujo de datos.
IA	inteligencia artificial.
JSON	JavaScript Object Notation.
KLOC	miles de líneas de código.
OWASP	Open Worldwide Application Security Project.
RF	requerimiento funcional.
RNF	requerimiento no funcional.
SARIF	Static Analysis Results Interchange Format.
SAST	Static Application Security Testing.
SQL	Structured Query Language.
TIR	tasa interna de retorno.
UX	User Experience.
VAN	valor actual neto.
XSS	Cross-Site Scripting.

Símbolos

E	Exposición de un riesgo, producto de su probabilidad por su impacto.
F_1	Media armónica entre precisión y exhaustividad.
$G = (V, E)$	Grafo dirigido con vértices V y aristas E .
S, K, F	Conjuntos de nodos source, sink y sanitizer del análisis de contaminación.
TP, FP, TN, FN	Verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.